

Individuelle Praktische Arbeit

Spielkader Webapplikation (Frontend) für Fussballverein (React & Next.JS)

Dieses Dokument enthält den Bericht zur individuellen praktischen Arbeit von Flurin Bandli.



Inhaltsverzeichnis

1.	Teil 1: Umfeld und Ablauf	5
1.1	Projektaufbauorganisation	5
1.1.1	Projektorganisation	5
1.1.2	Ausführungszeitraum	5
1.1.3	Termine	5
1.1.4	Involvierte Personen	5
1.2	Zeitplan	6
1.3	Arbeitsprotokoll	10
1.3.1	Montag, 02. März 2026	10
1.3.2	Dienstag, 03. März 2026	11
1.3.3	Mittwoch, 04. März 2026	12
1.3.4	Donnerstag, 05. März 2026	13
1.3.5	Freitag, 06. März 2026	14
1.3.6	Montag, 09. März 2026	15
1.3.7	Dienstag, 10. März 2026	16
1.3.8	Mittwoch, 11. März 2026	17
1.3.9	Donnerstag, 12. März 2026	18
1.3.10	Freitag, 13. März 2026	19
2.	Teil 2: Projekt	20
2.1	Kurzfassung	20
2.1.1	Ausgangssituation	20
2.1.2	Umsetzung	20
2.1.3	Ergebnis	20
2.2	Informieren	21
2.2.1	Verstandene Aufgabenstellung und Ziel der Arbeit	21
2.2.2	Ausgangslage	21
2.2.3	Abklärungen	25
2.2.4	Projektumfeld und Systemgrenzen	25
2.3	Planen	26
2.3.1	Verwendete Projektmanagementmethode	26

2.3.2	Versionierung und Datensicherheit	26
2.3.3	Priorisierung der Tätigkeiten.....	27
2.3.4	Use-Case Diagramm	28
2.3.5	Routen	29
2.3.6	Testkonzept	30
2.3.7	Erläuterung zum Zeitplan	33
2.4	Entscheiden	34
2.4.1	Beschreibung UI Library	34
2.4.2	Varianten	34
2.4.3	Entscheidung UI Library	35
2.4.4	Beschreibung UI-Interaktionen	35
2.4.5	Varianten	35
2.4.6	Entscheidung UI-Interaktionen	36
2.5	Realisieren	38
2.5.1	Abbildung des Gesamtsystems	38
2.5.2	Grundaufbau / Architektur des Projekts.....	38
2.5.3	Vorgehensweise	39
2.5.4	Datenbank	42
2.5.5	Anbindung an die Datenbank	42
2.5.6	Routen	43
2.5.7	Implementierung der Kernfeatures	44
2.5.8	Vorbereitungen für den Upload / Live-Schaltung / Implementierung	48
2.6	Kontrollieren	48
2.6.1	Beschreibung der Randbedingungen / Testanlage (Umfeld)	48
2.6.2	Testprotokoll	49
2.6.3	Bugfixing	56
2.7	Auswerten	57
2.7.1	Reflexion der Vorgehensweise	57
2.7.2	Bewertung des Produktes	57
2.7.3	Abweichungen zum Zeitplan	58
2.7.4	Persönliches Schlusswort und Bilanz.....	58
2.8	Glossar	60

2.9	Quellenverzeichnis	62
3.	Anhang	63
3.1	Git-Commit-History	63
3.2	Projektjournal	66
3.3	Erster Expertenbesuch	67
3.4	Zweiter Expertenbesuch	68
3.5	Code	69
3.5.1	lib/nest-api.ts	69
3.5.2	login-form.tsx	71
3.5.3	squad-builder.tsx	72
3.6	Zusätzliche Manuals, Skripts und weiteres	74

1. Teil 1: Umfeld und Ablauf

1.1 Projektaufbauorganisation

1.1.1 Projektorganisation

Auszubildender	Lehrbetrieb
Flurin Bandli Nussbaumstrasse 16 8003 Zürich	twofold academy AG Thurgauerstrasse 54 8050 Zürich

1.1.2 Ausführungszeitraum

Mo 02. März 2026	Di 03. März 2026	Mi 04. März 2026	Do 05. März 2026
Fr 06. März 2026	Mo 09. März 2026	Di 10. März 2026	Mi 11. März 2026
Do 12. März 2026	Fr 13. März 2026		

1.1.3 Termine

Was?	Wann?
1. Expertenbesuch	Di 03. März 2026 10:00 Uhr
2. Expertenbesuch	Mi 11. März 2026 13:00 Uhr
Präsentation, Demonstration, Fachgespräch	Mo 23. März 2026 10:30 Uhr

1.1.4 Involvierte Personen

Person	Rolle
Marius Schumacher	Hauptexperte
Duc-Lam Tran	Nebenexperte
René Bach	Verantwortliche Fachkraft & Berufsbildner
Flurin Bandli	IPA-Prüfungskandidat

1.2 Zeitplan

Der folgende Zeitplan zeigt die geplanten sowie die effektiv benötigten Stunden für die einzelnen Tätigkeiten während der IPA.

Arbeit	Aufwand (h)		Montag								Dienstag								Mittwoch							
			02.03.2026								03.03.2026								04.03.2026							
	Sollzeit	Istzeit	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8
Arbeitsjournal führen	4,00	4,00																								
Zeitplan erstellen	2,00	2,00																								
Dokumentation	14,00	16,00																								
Git Repository einrichten	1,00	1,00																								
Expertenbesuche	3,00	3,00																								
Informationen sammeln	1,00	1,00																								
Seitenstruktur planen	1,00	1,00																								
Priorisierung der Tätigkeiten	1,00	1,00																								
Use Case Diagramm erstellen	1,50	1,00																								
Route Map Diagramm erstellen	1,50	1,00																								
Testkonzept erstellen	2,00	2,00																								
Lösungsvariante festlegen	2,00	2,00																								
Seitenstruktur erstellen	1,00	1,00																								
Backend Verknüpfung anlegen	2,00	2,00																								
Authentifizierung mit NextAuth	4,00	4,00																								

Abbildung 1: Zeitplan des Projekts (Teil 1)

Quelle: Eigene Darstellung

Arbeit	Aufwand (h)		Donnerstag 05.03.2026							Freitag 06.03.2026							Montag 09.03.2026						
Dokumentation	14,00	16,00																					
Git Repository einrichten	1,00	1,00																					
Expertenbesuche	3,00	3,00																					
Informationen sammeln	1,00	1,00																					
Seitenstruktur planen	1,00	1,00																					
Priorisierung der Tätigkeiten	1,00	1,00																					
Use Case Diagramm erstellen	1,50	1,00																					
Route Map Diagramm erstellen	1,50	1,00																					
Testkonzept erstellen	2,00	2,00																					
Lösungsvariante festlegen	2,00	2,00																					
Seitenstruktur erstellen	1,00	1,00																					
Backend Verknüpfung anlegen	2,00	2,00																					
Authentifizierung mit NextAuth	4,00	4,00																					
Backend Layout kreieren	4,00	3,50																					
Backend Squad Bereich erstellen	9,00	11,00																					
Frontend Squad Bereich erstellen	5,00	5,50																					

Abbildung 2: Zeitplan des Projekts (Teil 2)

Quelle: Eigene Darstellung

Arbeit	Aufwand (h)		Dienstag 10.03.2026							Mittwoch 11.03.2026						
Expertenbesuche	3,00	3,00														
Informationen sammeln	1,00	1,00														
Seitenstruktur planen	1,00	1,00														
Priorisierung der Tätigkeiten	1,00	1,00														
Use Case Diagramm erstellen	1,50	1,00														
Route Map Diagramm erstellen	1,50	1,00														
Testkonzept erstellen	2,00	2,00														
Lösungsvariante festlegen	2,00	2,00														
Seitenstruktur erstellen	1,00	1,00														
Backend Verknüpfung anlegen	2,00	2,00														
Authentifizierung mit NextAuth	4,00	4,00														
Backend Layout kreieren	4,00	3,50														
Backend Squad Bereich erstellen	9,00	11,00														
Frontend Squad Bereich erstellen	5,00	5,50														
Backend Player Bereich erstellen	4,00	4,00														
Backend Trainer Bereich erstellen	2,00	2,00														

Abbildung 3: Zeitplan des Projekts (Teil 3)

Quelle: Eigene Darstellung

Arbeit	Aufwand (h)		Donnerstag 12.03.2026								Freitag 13.03.2026								IPERKA Planen
Use Case Diagramm erstellen	1,50	1,00																	
Route Map Diagramm erstellen	1,50	1,00																	
Testkonzept erstellen	2,00	2,00																	
Lösungsvariante festlegen	2,00	2,00																	Entscheiden
Seitenstruktur erstellen	1,00	1,00																	Realisieren
Backend Verknüpfung anlegen	2,00	2,00																	
Authentifizierung mit NextAuth	4,00	4,00																	
Backend Layout kreieren	4,00	3,50																	
Backend Squad Bereich erstellen	9,00	11,00																	
Frontend Squad Bereich erstellen	5,00	5,50																	
Backend Player Bereich erstellen	4,00	4,00																	
Backend Trainer Bereich erstellen	2,00	2,00																	
Testen	5,00	4,00																	Kontrollieren
Validierung & Fehlerbehebung	3,00	3,00																	
Reflexion & Fazit	3,00	3,00																	Auswerten
Dokumentation abschliessen	5,00	5,00																	
Total	81,00	83,00																	

Abbildung 4: Zeitplan des Projekts (Teil 4)

Quelle: Eigene Darstellung

1.3 Arbeitsprotokoll

1.3.1 Montag, 02. März 2026

Tagesziele gemäss Zeitplan	Zeitplan erstellen (2h), Dokumentation (1,5h), Git Repository einrichten (1h), Informationen sammeln (1h), Seitenstruktur planen (1h), Priorisierung der Tätigkeiten (1h)
Ungeplante Arbeiten / geleistete Überstunden	Keine
Erreichte Ziele, Erfolgserlebnisse	Anstehende Arbeiten im Zeitplan aufgeteilt, erste Teile der Projektdokumentation erstellt.
Herausforderungen, Probleme	Beim Einrichten eines Repository-Mirroring zwischen GitLab und GitHub gab es Probleme, da immer wieder eine Fehlermeldung auftrat und es nicht funktionierte. Repository-Mirroring bedeutet, dass Änderungen in einem Repository automatisch in ein zweites Repository übertragen werden. GitLab wird als primäres Repository verwendet, während GitHub als Backup dient. Dadurch wird sichergestellt, dass der Quellcode redundant abgespeichert ist.
Lösungen	Die offizielle Dokumentation von GitLab zum Thema Repository-Mirroring wurde konsultiert. Die Konfiguration wies keine offensichtlichen Fehler auf. Nach mehreren erneuten Versuchen funktionierte das Mirroring wie gewünscht und die Commits wurden erfolgreich synchronisiert.
Durchgeführte Tests	Es wurde überprüft, ob ein Commit auf GitLab auch automatisch auf GitHub synchronisiert wird. Das Mirroring funktioniert einwandfrei.
Wissensbeschaffung	https://docs.gitlab.com/user/project/repository/mirror/
Beanspruchte Hilfeleistung	Keine
Vergleich mit dem Soll-Zeitplan	Genau im Zeitplan
Persönliche Tagesreflexion	Obwohl ich mich sehr gut für die IPA vorbereitet habe, spürte ich in der Nacht und auch am Morgen auf dem Weg zur Arbeit eine deutliche Nervosität. Im Verlaufe des Tages legte sich meine Nervosität mehr und mehr. Ich konnte gut arbeiten und meine Tagesziele erreichen. Was ich aus dem heutigen Tag mitnehme, ist, dass eine gute Vorbereitung meine Nervosität zwar nicht entfernt, sie aber wesentlich lindert.

1.3.2 Dienstag, 03. März 2026

Tagesziele gemäss Zeitplan	Use-Case Diagramm (1,5h), Expertenbesuch (1h), Route Map Diagramm (1,5h), Testkonzept erstellen (2h), Lösungsvariante festlegen (2h)
Ungeplante Arbeiten / geleistete Überstunden	Keine
Erreichte Ziele, Erfolgserlebnisse	Zwei Diagramme erstellt, die beim Programmieren sehr hilfreich sein werden. Für das Testen ein ausführliches Konzept erstellt, ersten Expertenbesuch überstanden.
Herausforderungen, Probleme	Da für die Applikation kein Design vorgegeben war und die zur Verfügung stehende Zeit begrenzt ist, soll eine Design-Library verwendet werden. Design-Libraries stellen vorgefertigte Komponenten zur Verfügung, wodurch die Entwicklung von Benutzeroberflächen weniger Zeit in Anspruch nimmt. Für dieses Projekt wurden folgende drei Libraries in Betracht gezogen: Material UI, Ant Design und shadcn/ui.
Lösungen	Mithilfe einer Vergleichsmatrix wurden die drei Libraries verglichen und shadcn/ui als passende Lösung ausgewählt. Für genauere Informationen siehe 2.4.1 im Dokument.
Durchgeführte Tests	Keine
Wissensbeschaffung	Dokumentationen von Material UI und Ant Design angeschaut. https://mui.com/material-ui/ https://ant.design/components/overview/
Beanspruchte Hilfeleistung	KI als Hilfsmittel konsultiert, um die genauen Unterschiede zwischen shadcn/ui, Material UI und Ant Design herauszufinden.
Vergleich mit dem Soll-Zeitplan	Da ich für die zwei Diagramme und das Expertengespräch weniger Zeit als geplant in Anspruch genommen habe, bin ich gut eine Stunde vor dem Zeitplan und konnte das Erstellen der Seitenstruktur, welches für morgen geplant gewesen wäre, bereits erledigen.
Persönliche Tagesreflexion	Vor dem ersten Gespräch war ich sehr nervös, obwohl ich wusste, dass nichts Schlimmes passieren würde. Während dem Gespräch hingegen ging es mir gut und ich ging mit einem super Gefühl aus dem Gespräch hinaus. Ich bin sehr zufrieden mit meiner geleisteten Arbeit heute und war schneller als ich im Zeitplan vorgesehen habe. Mein Route Map Diagramm half mir enorm beim Erstellen meiner Routen.

1.3.3 Mittwoch, 04. März 2026

Tagesziele gemäss Zeitplan	Seitenstruktur erstellen (1h), Backend Verknüpfung anlegen (2h), Authentifizierung mit NextAuth (4h), Dokumentation (1h)
Ungeplante Arbeiten / geleistete Überstunden	Keine
Erreichte Ziele, Erfolgserlebnisse	Login eingebaut und geschützten Admin Bereich erstellt. Verbindung zum NestJS Backend erfolgreich hergestellt.
Herausforderungen, Probleme	Der Admin Bereich der Applikation darf nur von autorisierten Personen verwendet werden. Um dies zu gewährleisten, braucht es eine technische Lösung. Für die Umsetzung einer Authentifizierung kommen verschiedene Methoden in Frage, zum Beispiel eine eigene Implementierung zu erstellen oder bestehende Libraries zu nutzen. Während der Implementierung wurden in den Environment Variablen der Benutzername und das Passwort abgespeichert. Für den Benutzernamen wurde die Variable USERNAME verwendet, was sich als Problem herausstellte.
Lösungen	Für die Authentifizierung wurde die NextAuth Library verwendet. Sie ist speziell für Next.js entwickelt, ermöglicht eine rasche Integration und ist dem Entwickler am besten vertraut. Durch einen Credentials Provider werden Benutzername und Passwort in den Environment Variablen definiert und mit der Eingabe des Benutzers verglichen. Während der Implementierung zeigte sich ein Problem bei diesem Vergleich. Bei der Fehlerbehebung wurde identifiziert, dass der festgelegte Benutzername in der Environment Variable von Windows automatisch gesetzt wurde. Durch die Änderung der Variable von USERNAME auf AUTH_USERNAME konnte das Problem behoben werden.
Durchgeführte Tests	Login getestet, Fehlermeldungen bei ungültigem Login angeschaut, Verbindung zum Backend mit Console Log sichergestellt.
Wissensbeschaffung	Dokumentation von NextAuth als Unterstützung bei Einbau von Nextauth verwendet https://authjs.dev/getting-started/installation?framework=Next.js
Beanspruchte Hilfeleistung	KI gefragt, wieso sich USERNAME nicht ändern lässt. Windows setzt USERNAME automatisch als Systemvariable, welche sich nicht durch eine Environment Variable überschreiben lässt.
Vergleich mit dem Soll-Zeitplan	Etwa eine halbe Stunde vor dem Zeitplan. Ich brauchte bei der Implementierung von NextAuth und der Dokumentation etwas länger als geplant.
Persönliche Tagesreflexion	Super in den Tag gestartet. Ich hatte vorher etwas Angst, dass ich mehr Zeit für die Backend Verbindung benötigen würde, ich brauchte jedoch ungefähr genauso lange wie dafür vorgesehen. Bei NextAuth hatte ich ein Problem mit den Environment Variablen, was mich etwas Zeit gekostet hat. Dafür weiss ich nun, dass USERNAME sich nicht ändern lässt.

1.3.4 Donnerstag, 05. März 2026

Tagesziele gemäss Zeitplan	Backend Layout kreieren (4h), Dokumentation (2h), Backend Squad Bereich erstellen (2h)
Ungeplante Arbeiten / geleistete Überstunden	Keine
Erreichte Ziele, Erfolgserlebnisse	Die Navigation auf der linken Seite wurde implementiert und funktioniert wie geplant. Der Logout-Button wurde im Layout integriert. Die Übersichtsseite im Team Bereich wurde erstellt.
Herausforderungen, Probleme	Da keine genauen Vorgaben zur UI vorgegeben wurden, war unklar an welcher Stelle der Logout-Button platziert werden sollte. Es gab zwei Optionen: Entweder den Logout- Button oben rechts im Header platzieren oder den Logout-Button unten in der Sidebar Navigation zu platzieren.
Lösungen	Falls der Logout-Button im Header platziert wäre, müsste dieser entweder fixiert werden, oder der Benutzer müsste gegeben Falls nach oben scrollen. In der Sidebar Navigation würde dieses Problem nicht auftreten und der Button passt besser ins Erscheinungsbild. Aus diesen Gründen wird der Logout-Button in der Sidebar platziert.
Durchgeführte Tests	Was passiert, wenn das Backend nicht erreichbar ist mit der Teams Seite? Die Fehlermeldung «Backend nicht erreichbar» wird angezeigt.
Wissensbeschaffung	Die Dokumentation der shadcn/ui Komponente Sidebar wurde bei der Erstellung der Sidebar Navigation als Unterstützung verwendet. https://ui.shadcn.com/docs/components/radix/sidebar
Beanspruchte Hilfeleistung	Keine
Vergleich mit dem Soll-Zeitplan	Für das Backend Layout brauchte ich etwas weniger als die geplanten vier Stunden und da am Vortag der Vorsprung auf den Zeitplan etwa bei einer halben Stunde lag, liegt er jetzt bei ungefähr einer Stunde.
Persönliche Tagesreflexion	Ich war heute vor allem nach dem Mittag etwas erschöpft, da ich gestern Abend ein sehr schlechtes Fussballspiel ansehen musste. Mit der heutigen geleisteten Arbeit bin ich nichts destotrotz zufrieden.

1.3.5 Freitag, 06. März 2026

Tagesziele gemäss Zeitplan	Backend Squad Bereich (7h), Dokumentation (1h).
Ungeplante Arbeiten / geleistete Überstunden	Keine
Erreichte Ziele, Erfolgserlebnisse	Implementierung des Backend Squad Bereichs abgeschlossen. Teams können nun neu erstellt, bearbeitet und gelöscht werden.
Herausforderungen, Probleme	Eine Herausforderung stellte die Kommunikation mit dem Backend dar. Da die Authentifizierung über NextAuth erfolgt, können API-Requests nicht direkt aus der Client Komponente an das Backend gesendet werden. Als Lösung gab es entweder Server Actions oder API-Routen. Für das Hinzufügen und Bearbeiten von Spielern und Trainern musste entschieden werden, wie die UI funktionieren sollte.
Lösungen	Für Delete Squad wurde eine Server Action erstellt, da dies schneller und effizienter ist. Für PATCH und POST hingegen wurden API-Routen verwendet, da bei der Umsetzung mit Server Actions Probleme auftraten. Für die Auswahl von Spielern und Trainern wurde eine Combobox Komponente von shadcn/ui verwendet. Dies ermöglicht zum einen ein Suchfeld und zum anderen das Scrollen mit der Maus.
Durchgeführte Tests	Erstellung eines neuen Teams getestet. Bearbeiten eines Teams getestet. Löschen eines Teams getestet.
Wissensbeschaffung	Shadcn/ui Komponente Combobox konsultiert. API-Dokumentation des NestJS Backends verwendet, um die erwarteten Requests zu überprüfen.
Beanspruchte Hilfeleistung	KI-Unterstützung zur Analyse von Fehlern bei der Implementierung der API-Routen sowie bei der Datenzuordnung zwischen Backend-Antworten und Frontend-State verwendet.
Vergleich mit dem Soll-Zeitplan	Da der Backend Squad Bereich, der am schwierigsten zu implementierende Bereich ist, wurden neun Stunden dafür eingerechnet. Bei der Implementierung der API und der Combobox zur Auswahl von Spielern und Trainern wurde mehr Zeit gebraucht als geplant. Aktuell etwa eine Stunde hinter dem Zeitplan.
Persönliche Tagesreflexion	Heute kam ich zwar nicht so weit wie ich wollte, ich habe dafür aber viel gelernt und bin sehr stolz auf meine geleistete Arbeit heute und im Verlauf der Woche. Ich glaube mein Zeitplan war etwas optimistisch gesetzt für den Backend Squad Bereich. Es gibt noch viele kleine Verbesserungen, die ich mir aufgeschrieben habe. Ich fokussiere mich momentan jedoch bewusst darauf, dass nur die Kernfunktionen funktionieren und alles was «Nice to have» wäre, nur falls die Zeit noch dafür reicht, implementiert wird.

1.3.6 Montag, 09. März 2026

Tagesziele gemäss Zeitplan	Dokumentation (2h), Frontend Squad Bereich (5h), Dokumentation (1h)
Ungeplante Arbeiten / geleistete Überstunden	Keine
Erreichte Ziele, Erfolgserlebnisse	Das Frontend wurde in einer ersten Version grafisch umgesetzt. Die Spieler werden nach Position sortiert und auf einem Fussballfeld aufgezeigt. Trainer sowie die Ersatzbank werden auf der rechten Seite dargestellt. Die ID wird gehasht, damit sie nicht erratbar ist. Per Button lässt sie sich kopieren und teilen.
Herausforderungen, Probleme	Da keine klaren Vorgaben zur Darstellung des Frontends zur Verfügung stehen, war unklar wie das Team optimal visualisiert werden soll.
Lösungen	Oben werden Name und Datum des Teams dargestellt. Auf der linken Seite wird die Startaufstellung gezeigt und auf der rechten Seite die Trainer und Ersatzspieler. Der Link zum Teilen eines Teams befindet sich rechts unten. Als Hintergrundbild wurde ein Fussballfeld ausgewählt.
Durchgeführte Tests	Öffnen der öffentlichen Seite, ohne angemeldet zu sein, getestet. Kopieren des Links im Zwischenspeicher getestet.
Wissensbeschaffung	Die Library «hashids» zum hashen der IDs angeschaut. https://www.npmjs.com/package/hashids
Beanspruchte Hilfeleistung	Mit René Server Actions nochmals angeschaut. Alle Routen zu Server Actions gemacht, damit es einheitlich ist und weniger Code geschrieben werden muss.
Vergleich mit dem Soll-Zeitplan	Ungefähr anderthalb Stunden hinter dem Zeitplan.
Persönliche Tagesreflexion	Am Morgen etwas erschlagen, obwohl ich genug geschlafen habe. Beim Dokumentieren hatte ich deswegen etwas Mühe, konnte aber trotzdem meinen Zeitplan einhalten. Beim Frontend hatte ich vor allem beim Design grosse Mühe. Da keine Mockups vorliegen hatte ich nur ein grobes Bild im Kopf wie das Design genau aussehen sollte. Mit der ersten Version bin ich nur mässig zufrieden mit dem Aussehen. Die Funktionalität ist aber komplett in Ordnung, was wichtiger ist.

1.3.7 Dienstag, 10. März 2026

Tagesziele gemäss Zeitplan	Dokumentation (4h), Backend Player Bereich (4h)
Ungeplante Arbeiten / geleistete Überstunden	Keine
Erreichte Ziele, Erfolgserlebnisse	Im Admin Bereich auf der Spieler Seite können neue Spieler erstellt und vorhandene Spieler bearbeitet oder gelöscht werden.
Herausforderungen, Probleme	Beim Implementieren der Sheet Komponente aus der shadcn/ui Library gab es anfängliche Schwierigkeiten bei der Integration in die Seitenstruktur. Das Öffnen und Schliessen des Sheets mussten angepasst werden.
Lösungen	Durch das Studium der shadcn/ui Dokumentation sowie kleinere Tests mit der Komponente konnte die Integration Schritt für Schritt umgesetzt werden. Anschliessend wurde das Sheet erfolgreich für das Erstellen und Bearbeiten von Spielern verwendet.
Durchgeführte Tests	Neue Spieler erstellen getestet. Spieler bearbeiten getestet. Spieler löschen getestet.
Wissensbeschaffung	Aus der shadcn/ui Library die Komponenten Sidebar und Radio Group studiert.
Beanspruchte Hilfeleistung	Nach einem Gespräch mit René eine globale Prettier Config installiert.
Vergleich mit dem Soll-Zeitplan	Circa eine Stunde hinter dem Zeitplan.
Persönliche Tagesreflexion	Die Tage gehen schneller und schneller vorbei. Ich konnte heute zwar meine Tagesziele erreichen, war aber nicht so konzentriert bei der Arbeit wie an anderen Tagen. Obwohl ich genügend geschlafen habe, spürte ich am Nachmittag eine gewisse Schläfrigkeit. Nichtsdestotrotz fühle ich mich sehr zuversichtlich, meine IPA gut abschliessen zu können. Am späten Nachmittag hatte ich ein super Gespräch mit René und ich gehe guten Mutes nach Hause.

1.3.8 Mittwoch, 11. März 2026

Tagesziele gemäss Zeitplan	Dokumentation (4h), Backend Trainer Bereich (2h), Expertenbesuch (2h).
Ungeplante Arbeiten / geleistete Überstunden	Keine
Erreichte Ziele, Erfolgserlebnisse	Der Trainer Bereich wurde erfolgreich umgesetzt. Trainer können nun erstellt, bearbeitet und gelöscht werden. Der zweite Expertenbesuch fand statt, wobei der aktuelle Stand und die Prioritäten für die letzten Tage diskutiert wurden. Die grundlegende Funktionalität der Applikation ist vollständig umgesetzt.
Herausforderungen, Probleme	Da sich das Projekt dem Ende nähert, stellt sich die Frage, wo die Prioritäten für die letzten Tage des Projekts liegen.
Lösungen	Der Fokus wird bewusst auf das Überprüfen der Bewertungskriterien und auf die Vervollständigung der Dokumentation gelegt. Funktionale Erweiterungen oder Designanpassungen werden nur umgesetzt, sofern noch genügend Zeit dafür vorhanden ist.
Durchgeführte Tests	Neue Trainer erstellen getestet. Trainer bearbeiten getestet. Trainer löschen getestet.
Wissensbeschaffung	Keine
Beanspruchte Hilfeleistung	Keine
Vergleich mit dem Soll-Zeitplan	Circa eine Stunde hinter dem Zeitplan.
Persönliche Tagesreflexion	Vor dem Expertenbesuch war ich angespannt und unsicher ob und falls ja, wie ich mich dafür vorbereiten muss. Während des Gesprächs hatte ich einen sehr guten Eindruck von Herrn Schumacher und dem Stand meines Projekts. Ich gehe mit einem positiven Gefühl in die letzten zwei Tage des Projekts.

1.3.9 Donnerstag, 12. März 2026

Tagesziele gemäss Zeitplan	Testen (5h), Validierung & Fehlerbehebung (3h)
Ungeplante Arbeiten / geleistete Überstunden	Keine
Erreichte Ziele, Erfolgserlebnisse	Die Webapplikation wurde anhand der definierten Testfälle getestet. Alle grundlegenden Funktionen wie das Bearbeiten, Löschen und Erstellen von Spielern, Trainern und Teams wurden erfolgreich überprüft. An der Benutzeroberfläche wurden kleinere Verbesserungen vorgenommen, um die Bedienbarkeit zu erhöhen.
Herausforderungen, Probleme	Beim Bearbeiten und Erstellen von Teams kam es zu Platzproblemen in der Benutzeroberfläche, da die Aufstellung sehr viel Platz benötigt.
Lösungen	Das Formular wurde oberhalb der Aufstellung platziert, wodurch für die Aufstellung mehr Platz geschaffen wurde. Die Übersichtlichkeit der Seite wurde deutlich verbessert.
Durchgeführte Tests	Erstellung eines neuen Teams getestet Bearbeiten eines bestehenden Teams getestet Löschen eines Teams getestet Spieler erstellen, bearbeiten und löschen getestet Trainer erstellen, bearbeiten und löschen getestet Öffentliche Teamansicht getestet Login mit gültigen und ungültigen Zugangsdaten getestet
Wissensbeschaffung	Dokumentation von Next.js sowie der verwendeten UI Library konsultiert, um kleinere Anpassungen an der Benutzeroberfläche umzusetzen.
Beanspruchte Hilfeleistung	Rücksprache mit der VF bezüglich der kleineren Verbesserungen der Benutzeroberfläche.
Vergleich mit dem Soll-Zeitplan	Die geplanten Arbeiten konnten im vorgesehenen Zeitrahmen umgesetzt werden.
Persönliche Tagesreflexion	Die Testphase verlief ohne grössere Probleme, was mich sehr gefreut hat. Da ich während der Entwicklung meine Applikation immer wieder getestet habe, hatte ich heute genügend Zeit, meine Benutzeroberfläche weiter zu verbessern. Ich bin sowohl mit meinem Admin Bereich als auch mit dem öffentlichen Bereich sehr zufrieden.

1.3.10 Freitag, 13. März 2026

Tagesziele gemäss Zeitplan	Reflexion & Fazit (3h), Dokumentation abschliessen (5h)
Ungeplante Arbeiten / geleistete Überstunden	Keine
Erreichte Ziele, Erfolgserlebnisse	Die Dokumentation wurde vollständig fertiggestellt. Fehlende Kapitel wie Auswerten, Glossar und Quellenverzeichnis wurden ergänzt. Der vollständige Quellcode wurde als ZIP-Datei vorbereitet und das Git Repository mit einem Release-Tag versehen.
Herausforderungen, Probleme	Vor dem Abschluss des Projekts mussten noch verschiedene kleinere Details überprüft und angepasst werden. Das Projekt musste pünktlich auf PKOrg abgegeben werden.
Lösungen	Der Code wurde nochmals getestet und kleinere Verbesserungen vorgenommen, ohne funktionierende Teile zu verändern. Zum Schluss wurde das Projekt ein letztes Mal gebaut und getestet, um sicherzustellen, dass alles korrekt funktioniert.
Durchgeführte Tests	Alle zentralen Funktionen der Applikation wurden nochmals manuell getestet.
Wissensbeschaffung	Keine
Beanspruchte Hilfeleistung	Feedback meiner VF zum Code wurde implementiert.
Vergleich mit dem Soll-Zeitplan	Projekt pünktlich abgegeben.
Persönliche Tagesreflexion	Ich wusste, dass ich das Projekt pünktlich und in einem guten Zustand abgeben werde. Trotzdem war ich heute nervöser als an den vorherigen Tagen. Schlussendlich konnte ich die IPA sauber abschliessen und alle Daten rechtzeitig auf PKOrg hochladen. Ich gehe mit einem mulmigen Gefühl ins Wochenende. Einerseits bin ich froh und stolz, dass ich die IPA erfolgreich abschliessen konnte. Andererseits steht noch der mündliche Teil bevor und ich merke, dass ich mich während den letzten zwei Wochen sehr intensiv mit dem Projekt beschäftigt habe und die Arbeit daran bereits etwas vermisse.

2. Teil 2: Projekt

2.1 Kurzfassung

Der Fussballklub FC Zürich-Leutschenbach möchte für seine Mitglieder eine Webapplikation bereitstellen, mit der Fussballteams grafisch dargestellt und über einen Link geteilt werden können. Das Backend zur Verwaltung der Daten wurde bereits von einem Mitarbeiter der twofold AG entwickelt und stellt die notwendigen Schnittstellen zur Verfügung.

2.1.1 Ausgangssituation

Der Auftrag lautet, eine Webapplikation zur Verwaltung und Darstellung von Fussballteams zu entwickeln. Es soll ein Admin Account erstellt werden, mit dem Teams verwaltet werden können. Neue Teams können erstellt sowie bestehende Teams betrachtet, bearbeitet oder gelöscht werden.

Eine Registrierung ist nicht notwendig. Der Zugriff auf den Admin Bereich erfolgt ausschliesslich über einen Admin Account. Jedes Team erhält einen eigenen Link, über den ein Team angeschaut und geteilt werden kann. Änderungen an den Daten sind über diesen Link jedoch nicht möglich.

2.1.2 Umsetzung

Das Projekt soll mithilfe des Next.js Frameworks, basierend auf der React Library, erstellt werden. Next.js eignet sich hervorragend für das Erstellen einer modernen Webanwendung. Für die UI soll, immer wenn möglich, eine UI Library verwendet werden.

Die Seitenstruktur wird in zwei Bereiche unterteilt: Einen öffentlichen und einen geschützten Admin Bereich.

Im öffentlichen Bereich befinden sich das Login sowie die öffentliche Ansicht einzelner Teams. Nach erfolgreichem Login wird der Benutzer automatisch in den geschützten Bereich weitergeleitet. Bei ungültigem Login erscheint eine Fehlermeldung und der Benutzer bleibt auf der Login Seite.

Im geschützten Bereich kann zwischen den drei Seiten Teams, Spieler und Trainer navigiert werden. Auf diesen Seiten werden die jeweiligen Daten aufgelistet und können erstellt, bearbeitet oder gelöscht werden.

2.1.3 Ergebnis

Als Ergebnis entsteht eine funktionsfähige Webapplikation, mit der Fussballteams grafisch dargestellt werden können. Über einen geschützten Admin Bereich können Teams, Spieler und Trainer verwaltet werden. Ohne Login können ausschliesslich die Teamdarstellungen aufgerufen werden. Die Teamdarstellungen können über einen Link öffentlich aufrufen und einfach geteilt werden.

2.2 Informieren

In diesem Kapitel werden die Ausgangslage des Projekts, erste Abklärungen sowie das Projektumfeld beschrieben.

2.2.1 Verstandene Aufgabenstellung und Ziel der Arbeit

Ziel dieser Arbeit ist es, eine Webapplikation zur Darstellung und Verwaltung von Fussballteams für den FC Zürich-Leutschenbach zu dokumentieren und umzusetzen.

In dieser Arbeit wird ein auf Next.js basiertes Frontend entwickelt, das an ein bestehendes NestJS basiertes Backend angebunden wird. Teams sollen visuell dargestellt und über eine eindeutige URL aufgerufen werden können. In einem geschützten Bereich können Änderungen an den Teams, Spielern und Trainern vorgenommen werden.

2.2.2 Ausgangslage

Für die Planung der Benutzeroberfläche wurden erste Mockups in Figma erstellt. Die folgenden Mockups zeigen die geplante Struktur des Admin Bereichs, sowie die Darstellung der Teams und Spieler. Die Mockups dienen als Orientierung für die spätere Umsetzung.

Dieses Mockup zeigt die geplante Übersicht der Teams im Admin Bereich. Alle vorhandenen Teams sollen aufgelistet werden. Es besteht die Möglichkeit Teams anzuschauen zu bearbeiten oder zu löschen.

Admin Bereich

Logout

Team

Spieler

Trainer

☐	Name ↑↓	Beschreibung ↑↓	Datum	Table heading		
☐	Name	Table cell	13.01.2026	View	Edit	Delete
☐	Name	Table cell	Table cell	View	Edit	Delete
☐	Name	Table cell	Table cell	View	Edit	Delete
☐	Name	Table cell	Table cell	View	Edit	Delete
☐	Name	Table cell	Table cell	View	Edit	Delete
☒	Name	Table cell	Table cell	View	Edit	Delete
☐	Name	Table cell	Table cell	View	Edit	Delete

Abbildung 5: Mockup der Teamübersicht im Admin Bereich

Quelle: Eigene Darstellung (Figma)

Admin Bereich

Logout

Team

Spieler

Trainer

ID

FCZ - FCB

Der Klassiker

May 20, 2025

Trainer:

Ersatzbank:

twofold academy ag | Thurgauerstrasse 54 | 8050 Zürich | hi@twofold.swiss | twofold.swiss

Dieses Mockup zeigt die Übersicht der Spieler im Admin Bereich. Alle vorhandenen Spieler werden aufgelistet und können bearbeitet oder gelöscht werden. Des Weiteren besteht die Möglichkeit neue Spieler zu erstellen.

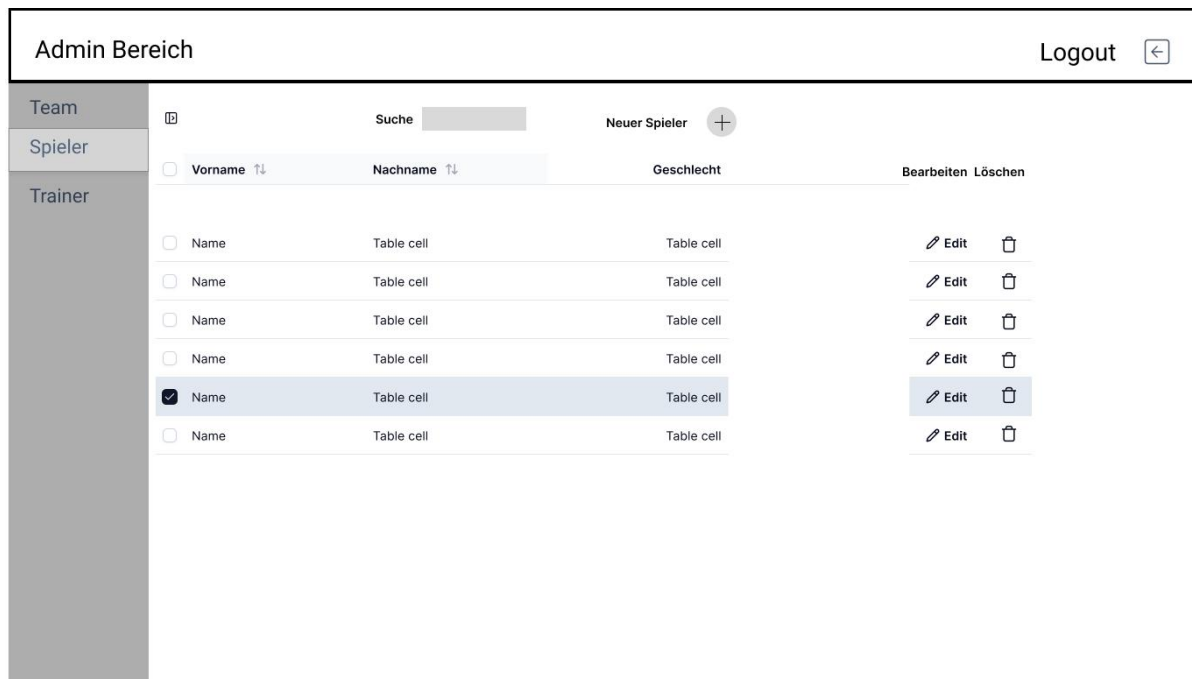


Abbildung 7: Mockup der Spielerübersicht im Admin Bereich

Quelle: Eigene Darstellung (Figma)

Dieses Mockup zeigt die Bearbeitungsansicht eines Spielers. Auf der rechten Seite öffnet sich ein Sheet, wo die persönlichen Daten eines Spielers angepasst werden können.

The mockup displays an 'Admin Bereich' interface. A sidebar on the left contains 'Team', 'Spieler' (selected), and 'Trainer'. The main content area features a table with columns 'Vorname', 'Nachname', and 'Geschlecht'. The table lists several players, with 'Max' highlighted. To the right, a 'Spieler bearbeiten' modal is open, containing form fields for 'Vorname' (Max), 'Nachname' (Muster), and 'Geschlecht' (a dropdown menu). At the bottom of the modal are 'Speichern' and 'Abbrechen' buttons.

Abbildung 8: Mockup zum Bearbeiten eines Spielers

Quelle: Eigene Darstellung (Figma)

Die Mockups dienten während der Umsetzung als Orientierung und halfen dabei, die Benutzeroberfläche strukturiert zu planen.

2.2.3 Abklärungen

Vor der Umsetzung der Applikation mussten einige Punkte genauer angeschaut werden.

Welche UI Library soll für die Benutzeroberfläche verwendet werden?

Die möglichen Varianten sind Material UI, Ant Design und shadcn/ui.

Wie soll die Benutzeroberfläche im Administratorbereich gestaltet werden?

Die möglichen Varianten sind Formulare alle auf separaten Seiten, nur die Teams Seiten auf separaten Seiten oder alles als Modals.

2.2.4 Projektumfeld und Systemgrenzen

Das Projekt wird lokal aufgesetzt. Für das Frontend wird Next.js verwendet. Das Backend existiert bereits und ist mit NestJS umgesetzt. Die Datenbank wird ausschliesslich durch das Backend angesprochen. Das Frontend kommuniziert über definierte API-Endpunkte mit dem Backend.

Die Umsetzung des Backends, sowie die Datenbankstruktur gehören nicht zum Umfeld dieses Projekts. Der Fokus liegt ausschliesslich auf dem Frontend, sowie der Anbindung an das bereits vorhandene Backend.

Eine Live-Schaltung oder ein produktiver Betrieb sind nicht Teil des Projekts.

2.3 Planen

In der Planungsphase werden die verwendete Projektmanagementmethode, das Testkonzept, die Datensicherheit sowie die Grundlagen für die Umsetzung des Projekts definiert.

2.3.1 Verwendete Projektmanagementmethode

Für die Umsetzung des Projekts wird die in der Schweiz weit verbreitete Projektmanagementmethode IPERKA verwendet. Sie besteht aus sechs aufeinanderfolgenden Phasen und ermöglicht eine strukturierte Planung, Umsetzung und Dokumentation eines Projekts.

Informieren:

In dieser Phase werden Informationen gesammelt und der Auftrag analysiert.

Planen:

In dieser Phase werden die Tätigkeiten priorisiert sowie die notwendigen Diagramme und das Testkonzept vorbereitet.

Entscheiden:

Verschiedene Lösungsvarianten werden miteinander verglichen und die geeignetste Lösung wird ausgewählt.

Realisieren:

Die geplanten Funktionalitäten der Applikation werden Schritt für Schritt umgesetzt.

Kontrollieren:

Die umgesetzten Funktionen werden mithilfe des Testkonzepts überprüft und allfällige Fehler werden behoben.

Auswerten:

Am Ende des Projekts wird das Vorgehen reflektiert und ein Fazit gezogen.

2.3.2 Versionierung und Datensicherheit

Bei jedem Commit auf unser firmeninternes GitLab wird der Commit automatisch auch auf mein privates GitHub gespiegelt. Dadurch wird Datenredundanz sichergestellt und ein möglicher Datenverlust vermieden. Des Weiteren wird die aktuelle Version der Dokumentation jeden Abend auf Dropbox hochgeladen.

Für jedes neue Feature wird ein neuer Branch erstellt. Nach Abschluss eines Features wird der Branch mit dem develop Branch gemerged. An jedem Abend wird der aktuelle Stand des Projekts in GitLab und GitHub abgespeichert.

Die vollständige Git Commit History wird im Anhang dieser Dokumentation dargestellt.

2.3.3 Priorisierung der Tätigkeiten

Die einzelnen Tätigkeiten des Projekts wurden nach ihrer Wichtigkeit priorisiert.

Beschreibung der Tätigkeit	Priorität
Zeitplan erstellen	Hoch
Dokumentation	Hoch
Git Repository einrichten	Mittel
Informationen sammeln	Hoch
Seitenstruktur planen	Mittel
Priorisierung der Tätigkeiten	Tief
Use-Case Diagramm	Hoch
Route Map Diagramm	Mittel
Testkonzept erstellen	Mittel
Lösungsvariante festlegen	Tief
Seitenstruktur erstellen	Hoch
Backend Verknüpfung anlegen	Mittel
Authentifizierung mit NextAuth	Hoch
Backend Layout kreieren	Mittel
Backend Squad Bereich erstellen	Hoch
Frontend Squad Bereich erstellen	Hoch
Backend Player Bereich erstellen	Mittel
Backend Trainer Bereich erstellen	Mittel
Testen	Hoch
Validierung & Fehlerbehebung	Hoch
Reflexion & Fazit	Hoch
Dokumentation abschliessen	Hoch

Die Priorisierung half dabei, die wichtigsten Funktionalitäten der Applikation zuerst umzusetzen und den Projektfortschritt effizient zu steuern.

2.3.4 Use-Case Diagramm

Das Use-Case Diagramm zeigt alle Interaktionen der beiden Rollen: dem Admin und dem Viewer. Der Viewer kann über eine öffentliche URL ein Team ansehen, hat jedoch keine Möglichkeit Änderungen vorzunehmen.

Der Admin kann sich über die Login Seite authentifizieren und erhält danach Zugriff auf den geschützten Admin Bereich. Dort können Teams, Spieler und Trainer verwaltet werden.

Das Diagramm zeigt somit die wichtigsten Funktionen der Applikation sowie die unterschiedlichen Berechtigungen der Beiden Benutzerrollen.

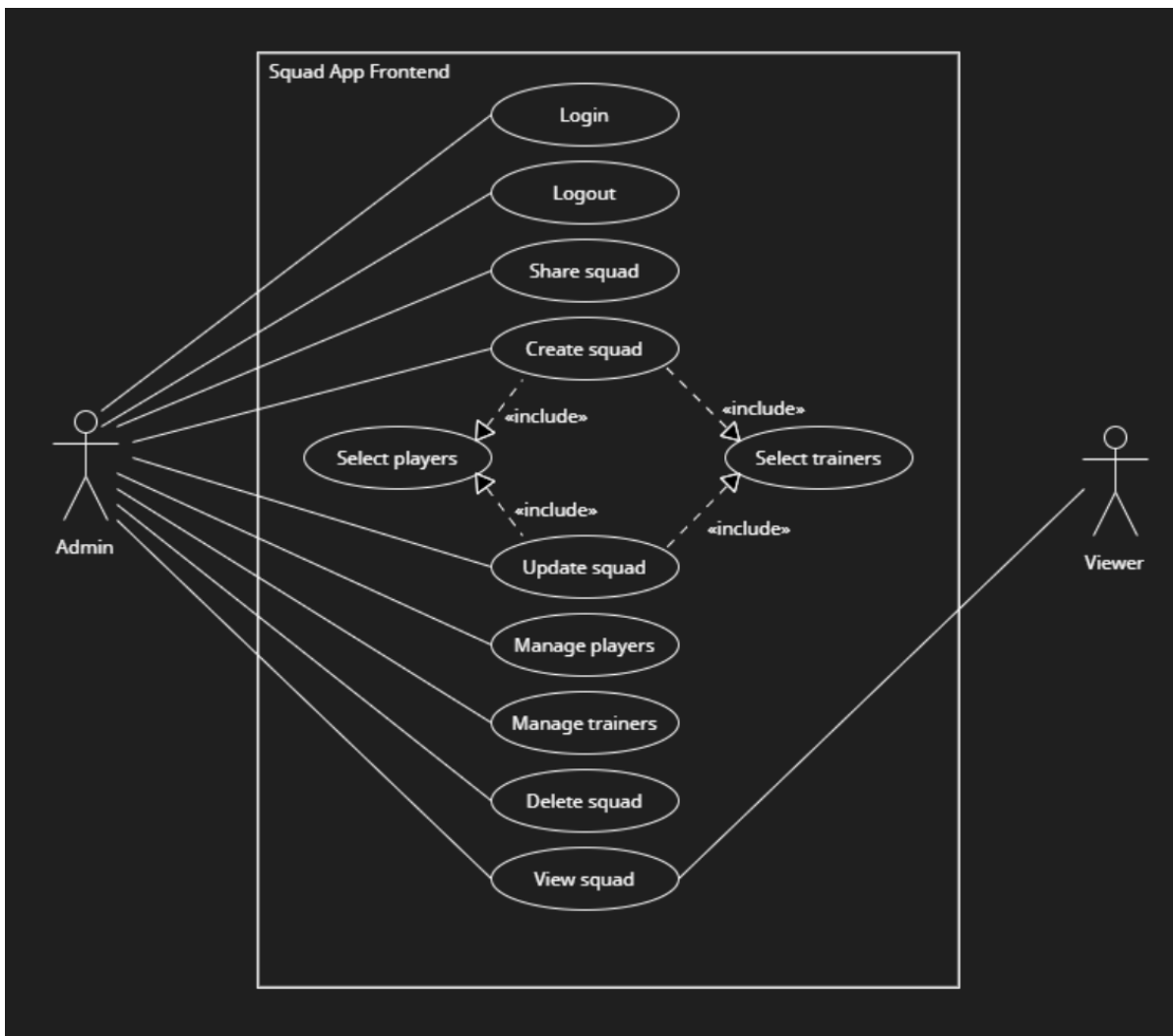


Abbildung 9: Use-Case Diagramm
Quelle: Eigene Darstellung (UMLet)

2.3.5 Routen

Das Route Map Diagramm zeigt die wichtigsten Seiten der Applikation. Die Routen sind in einen öffentlichen und einen geschützten Admin Bereich aufgeteilt

Öffentliche Benutzer können sich über das Login anmelden oder über eine eindeutige URL ein Team ansehen.

Nach erfolgreichem Login gelangt der Administrator in den geschützten Admin Bereich. Dort können Teams, Spieler und Trainer verwaltet werden.

Das Diagramm zeigt somit die grundlegende Navigationsstruktur der Applikation.

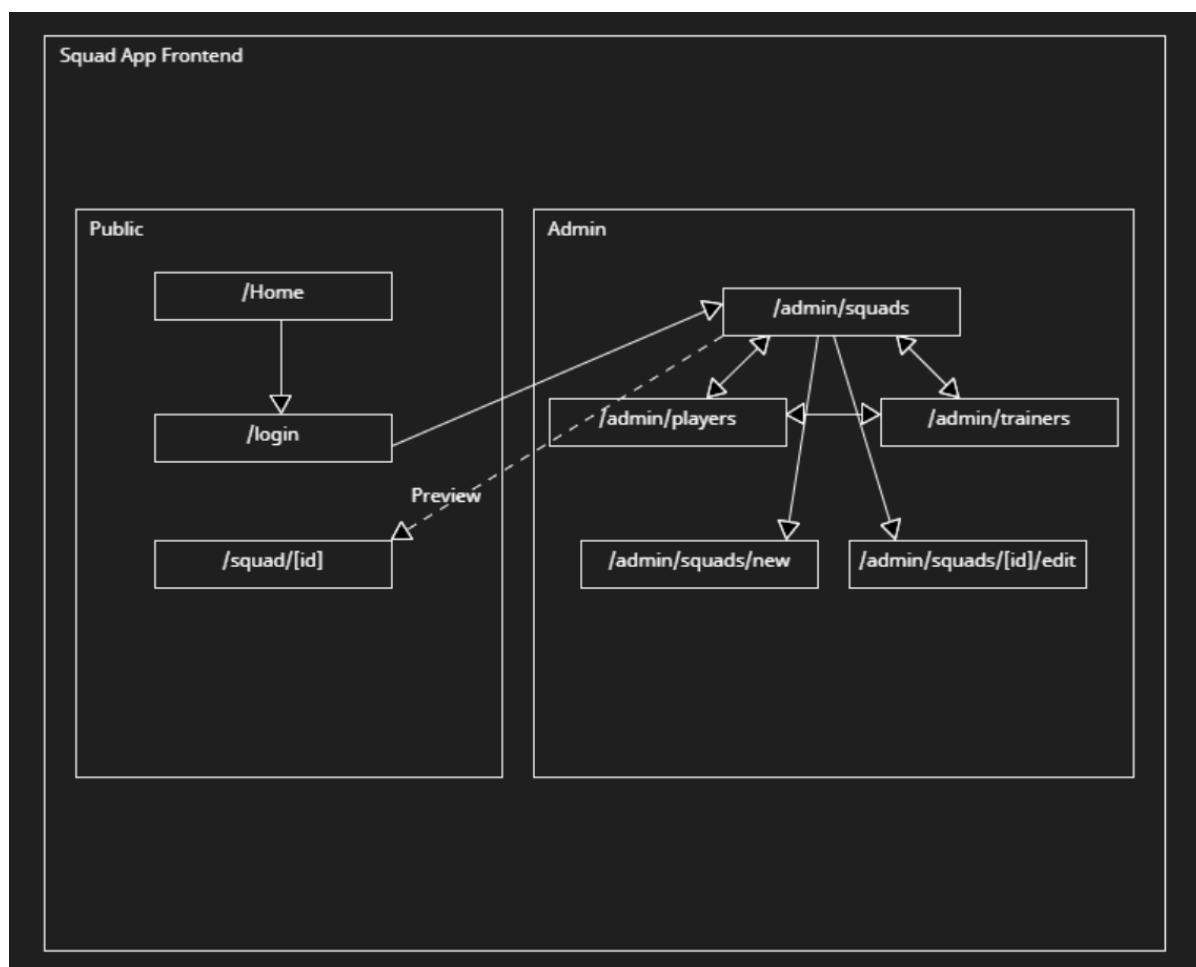


Abbildung 10: Route Map Diagramm

Quelle: Eigene Darstellung (UMLet)

2.3.6 Testkonzept

In diesem Kapitel wird beschrieben wie die Applikation getestet wird und welche Testfälle für die Überprüfung der wichtigsten Funktionen definiert werden.

2.3.6.1 Testmethode

Das Testen wird von Hand durchgeführt. Es wird ausschliesslich als Blackbox getestet. Der Quellcode wird nicht explizit angeschaut, sondern die Funktionen werden Schritt für Schritt ausprobiert und der Istzustand mit dem Sollzustand verglichen. Da Entwickler und Tester in diesem Projekt dieselbe Person sind, ist eine vollständige Trennung nicht möglich. Die Tests werden jedoch bewusst aus Anwendersicht durchgeführt.

2.3.6.2 Testmittel

Im Projekt gibt es einen Admin Account, welcher für das Login verwendet wird.

Für die Tests wurden folgende Spezifikationen verwendet:

Software:

- Windows 11 Pro 26200.7623
- Next.js v16.1.6
- Node.js v20
- NestJS Backend (bestehendes Projekt)
- Google Chrome v144
- Laragon für Datenbank

Hardware:

- CPU: Intel i7-12700H
- GPU: RTX 3060 Laptop Version
- RAM: 16 GB

Folgende Voraussetzungen sind für das Testen notwendig:

1. Repository klonen und Dependencies installieren mit: `npm install`
2. NestJS Backend und die Datenbank starten: Datenbank in Laragon starten. Im Backend `.env` konfigurieren.
3. Frontend starten und `.env` konfigurieren.
4. Applikation im Browser öffnen (<http://localhost:3000>)
5. Folgendes Login eingeben: Name: admin Passwort: admin

2.3.6.3 Abgrenzung

Getestet werden die im Projektauftrag definierten Kernfunktionen: Login/Logout, CRUD von Teams, Spieler und Trainer und die öffentliche Ansicht. Die Datenbank und das Backend werden nicht getestet, lediglich dass das Verwalten von Daten funktioniert. Die Webapplikation wird ausschliesslich auf Desktop Grösse getestet und nicht auf mobilen Geräten oder Tablets. Dies da Mobile und Tablet Versionen nicht explizit gefordert sind und die Zeit dafür zu knapp ist.

2.3.6.4 Testfälle

Folgende Tests wurden definiert und werden im Testen angeschaut:

#	Testfall	Vorbedingung	Vorgehen, Eingaben	Erwartetes Resultat
1	Login erfolgreich	Admin Passwort und Name vorhanden	Auf /login gültige Daten eingeben, auf Login klicken.	Weiterleitung auf /admin/squads
2	Login fehlgeschlagen	Keine	Auf /login ungültige Daten eingeben, auf Login klicken.	Keine Weiterleitung, sichtbare Fehlermeldung
3	Unbefugter Zugriff auf geschützten Bereich	Benutzer ist ausgeloggt	URL /admin/squads direkt öffnen	Weiterleitung auf /login
4	Logout	Benutzer ist eingeloggt	Auf Logout klicken	Session beendet, Weiterleitung auf /login
5	Trainer hinzufügen	Benutzer ist eingeloggt	/admin/trainers → «Neuer Trainer» → rechts Daten eingeben → Speichern	Trainer erscheint in der Mitte in der Liste aller Trainer
6	Spieler löschen	Benutzer ist eingeloggt	/admin/players → Abfall Icon → Bestätigen	Spieler wird aus der Liste gelöscht
7	Team bearbeiten	Team existiert und ist in /admin/squads sichtbar	/admin/squads → «Bearbeiten» → Spieler ändern → Speichern	Änderungen sind in Übersicht und öffentlicher Ansicht sichtbar und korrekt gespeichert

8	Share Link funktioniert	Team existiert und ist in /admin/squads sichtbar	/admin/squads → Auge Icon → Button «Link Kopieren»	Öffentliche Ansicht entspricht dem gespeicherten Team
9	Pflichtfelder Validierung	Benutzer ist eingeloggt	Spieler / Trainer ohne Pflichtfelder speichern	Speichern verhindert, Fehlermeldung erscheint
10	Gelöschter Spieler in bestehendem Team	Team existiert und enthält mindestens einen Spieler	Einen Spieler löschen, der in einem bestehenden Team verwendet wird. Danach die Teamübersicht öffnen.	Der gelöschte Spieler wird im Team nicht mehr angezeigt. Die restliche Darstellung des Teams funktioniert weiterhin korrekt.
11	Gelöschter Trainer in bestehendem Team	Team existiert und enthält mindestens einen Trainer	Einen Trainer löschen, der in einem bestehenden Team verwendet wird. Danach die Teamübersicht öffnen.	Der gelöschte Trainer wird im Team nicht mehr angezeigt. Die restliche Darstellung des Teams funktioniert weiterhin korrekt.
12	Lighthouse Performance Test	Applikation läuft lokal	Seite in Google Chrome öffnen → Lighthouse Analyse starten	Performance Score > 90%, Accessibility > 90%, Best Practices > 90%, SEO kann tiefer sein

2.3.7 Erläuterung zum Zeitplan

Der Zeitplan ist übersichtlich aufgebaut und zeigt sowohl die geplanten als auch die effektiv benötigten Stunden für die einzelnen Tätigkeiten. Jeder Arbeitsschritt wurde einer der sechs IPERKA Phasen zugeordnet.

2.4 Entscheiden

In diesem Kapitel werden wichtige technische und gestalterische Entscheidungen für die Umsetzung der Applikation beschrieben und begründet.

2.4.1 Beschreibung UI Library

Um eine konsistente Benutzeroberfläche zu gewährleisten und die Entwicklungszeit zu reduzieren, soll eine UI Library für möglichst viele Komponenten verwendet werden. Es gibt eine grosse Auswahl an UI Libraries. Drei davon werden hier verglichen und eine davon für das Projekt ausgewählt.

2.4.2 Varianten

Die folgenden Varianten wurden miteinander vergliche.

Variante A: Material UI

Vorteile:

- Sehr umfangreiche Komponenten Library
- Viele vorgefertigte Patterns

Nachteile:

- Tailwind Integration komplexer
- Anpassung kann aufwendig sein

Variante B: Ant Design

Vorteile:

- Konsistentes Designsystem
- Viele Komponenten sofort einsetzbar

Nachteile:

- Tailwind Integration komplexer
- Anpassung kann aufwendig sein

Variante C: shadcn/ui

Vorteile:

- Sehr gute Integration in Tailwind/Next.js
- Hohe Design-Flexibilität, da die Komponenten im eigenen Code liegen

Nachteile:

- Verantwortung für Updates/Anpassungen liegt stärker beim Projekt
- Nicht alles vorgefertigt, mehr Eigenleistung nötig

2.4.3 Entscheidung UI Library

Kriterium	Material UI	Ant Design	shadcn/ui
Tailwind Integration	Mittel	Mittel	Sehr gut
Design Flexibilität	Mittel	Mittel	Sehr gut
Komponenten Umfang	Sehr gut	Sehr gut	Mittel
Anpassbarkeit	Mittel	Mittel	Sehr gut
Gesamtbewertung	Gut	Gut	Sehr gut

Variante C: shadcn/ui wird verwendet. Der Hauptgrund ist die optimale Integration mit Tailwind CSS und Next.js. Die höhere Flexibilität beim Design ist ein weiterer Vorteil. Die Komponenten von shadcn/ui werden als Code ins Projekt übernommen und können bei Bedarf leicht angepasst und erweitert werden. Shadcn/ui ermöglicht eine effiziente Umsetzung der UI innerhalb des vorgegebenen Zeitrahmens.

2.4.4 Beschreibung UI-Interaktionen

Die Verwaltung von Teams benötigt deutlich mehr Bildschirmfläche als die Verwaltung von Spieler und Trainer. Ein kleines Formular genügt nicht, sondern es braucht ausserdem ein Spielfeld auf dem die Spieler der gewünschten Position (Torwart, Verteidigung, Mittelfeld, Sturm) hinzugefügt werden können. Aus diesem Grund muss entschieden werden, was wie umgesetzt werden sollte.

2.4.5 Varianten

Für die Umsetzung der Benutzerinteraktionen wurde folgende Varianten verglichen.

Variante A: Alles auf separaten Seiten

Vorteile:

- Gute Übersicht unabhängig von der Komplexität
- Einfach zu verlinken und klare Navigation

Nachteile:

- Höherer Aufwand bei der Implementierung
- Häufige Seitenwechsel

Variante B: Spieler / Trainer im Sheet, Team auf separater Seite

Vorteile:

- Gute Balance zwischen Übersicht und Effizienz

- Effiziente Bedienung
- Ausreichend Platz für Team Verwaltung

Nachteile:

- Unterschiedliche Darstellung zwischen Team und Spieler / Trainer
- Etwas komplexere Navigation

Variante C: Alles in Modals

Vorteile:

- Schnelle Bedienung
- Einheitliches Design

Nachteile:

- Zu wenig Platz und unübersichtlich
- Unhandlich bei vielen Inputs

2.4.6 Entscheidung UI-Interaktionen

Kriterium	Alles Seiten	Sheet + Teams Seite	Alles Modals
Übersicht	Sehr gut	Sehr gut	Schlecht
Bedienungseffizienz	Mittel	Sehr gut	Gut
Aufwand	Mittel	Gut	Mittel
Platz für Spielfeld	Sehr gut	Sehr gut	Schlecht
Gesamtbewertung	Gut	Sehr gut	Mittel

Variante B: Spieler / Trainer im Sheet, Team auf separater Seite wird umgesetzt. Die Spieler und Trainer werden über ein Sheet, das auf der rechten Seite aufgeht, verwaltet. Die Teams werden auf einer eigenen Seite verwaltet. Die Teams erhalten genügend Platz, sodass das Spielfeld gross genug ist, während Spieler und Trainer nicht unnötig viel Platz einnehmen. Zusätzlich bleibt die Umsetzung innerhalb des geplanten Zeitrahmens.

2.5 Realisieren

In diesem Kapitel wird beschrieben, wie die in der Planungsphase definierten Konzepte und Entscheidungen technisch umgesetzt werden.

2.5.1 Abbildung des Gesamtsystems

Die folgende Abbildung zeigt das Gesamtsystem. Das Frontend wurde mit Next.js erstellt und kommuniziert über eine API mit dem bestehenden NestJS Backend. Die Authentifizierung erfolgt über NextAuth. Nach erfolgreichem Login wird eine Session erstellt, welche als Cookie im Browser gespeichert wird.

Für die Kommunikation mit dem NestJS Backend wird zusätzlich ein Access Token verwendet, welcher vor jeder API-Anfrage automatisch angefordert und im Authorization Header mitgegeben wird.

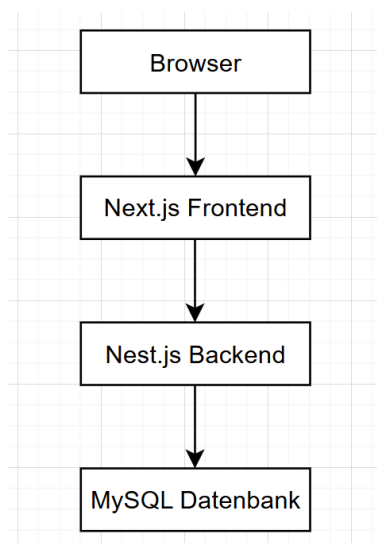


Abbildung 11: Architekturübersicht des Gesamtsystems

Quelle: Eigene Darstellung

2.5.2 Grundaufbau / Architektur des Projekts

Der Grundaufbau des Projekts basiert auf dem Next.js App Router. Die Applikation ist in verschiedene Bereiche unterteilt, um eine klare Struktur zu gewährleisten.

Die wichtigsten Ordner des Projekts sind:

- **app:** Enthält die Seiten der Applikation sowie die Routing Struktur.
- **components:** Wiederverwendbare UI-Komponenten aus shadcn/ui Library.
- **lib:** zentrale Funktionen, beispielsweise für die Kommunikation mit dem Backend.
- **Types:** Gemeinsame TypeScript-Typdefinition.

2.5.3 Vorgehensweise

Backend Verknüpfung anlegen:

Beim Studieren des zur Verfügung gestellten Backends wurde festgestellt, dass die Verbindung ein Login benötigt, welches einen Bearer Token zurückgibt. Dieser Token muss bei jeder Anfrage im Authorization Header mitgesendet werden.

Der Token könnte entweder bei jeder Anfrage neu erstellt werden, oder es könnte vor jeder Anfrage überprüft werden, ob ein gültiger Token bereits abgespeichert wurde. Bei jeder Anfrage einen neuen Token zu implementieren, wäre die einfachere Methode, es würden jedoch mehr Anfragen abgesendet werden. Da die öffentlichen Teams Seiten jedoch keinen Token benötigen, sondern nur die geschützten Seite, wurde die einfachere Methode gewählt.

Aus diesem Grund wurde die Datei «lib/nest-api.ts» erstellt, welche zwei Funktionen beinhaltet. Die erste Funktion übernimmt die Authentifizierung mit der NestJS API. Dabei wird ein Access Token vom Backend angefordert.

Die zweite dient als zentrale Methode für API-Anfragen. Vor jeder zukünftigen Anfrage an das Backend wird diese Funktion aufgerufen. Vor jeder Anfrage wird automatisch ein gültiger Access Token von der NestJS API geholt und im Authorization Header mitgegeben.

Authentifizierung mit NextAuth:

Für den geschützten Admin Bereich der Applikation wurde eine Authentifizierung mit der NextAuth Library umgesetzt. NextAuth ermöglicht eine einfache Integration von Login Mechanismen.

Für das Login wurde ein Credentials Provider eingerichtet, welcher die eingegebenen Daten des Benutzers mit den Daten aus den Environment Variablen vergleicht. Bei erfolgreicher Anmeldung wird eine Session erstellt und als Cookie abgespeichert.

Die Login Seite verwendet ein Formular welches React Hook Form zur Formularverwaltung und Zod zur Validierung der Eingaben verwendet.

Bei erfolgreicher Anmeldung wird der User in den geschützten Admin Bereich weitergeleitet. Bei ungültiger Eingabe bleibt der User auf der Login Seite. Nicht authentifizierte Benutzer werden automatisch auf die Login Seite umgeleitet.

Backend Layout kreieren:

Um den verschiedenen Admin Seiten eine einheitliche Struktur zu geben, wurde ein zentrales Layout für den Admin Bereich erstellt. Mit der Datei «layout.tsx» müssen die Elemente Navigation und Header nicht auf jeder Admin Seite, sondern nur einmal erstellt und implementiert werden.

Für die Navigation wurde die Komponente «Sidebar» aus der shadcn/ui Library verwendet. Die Sidebar Navigation ermöglicht den Seitenwechsel zwischen Teams, Spieler und Trainer.

Der Header zeigt die aktuell geöffnete Seite des Admin Bereiches und das Vereinslogo.

Backend Squad Bereich erstellen:

In einem ersten Schritt wurde die Übersichtsseite des Squad Bereiches erstellt. Zunächst werden vom NestJS Backend alle Teams abgefragt und in einer Variable abgespeichert. Als nächstes wurde mithilfe der Komponente Table aus der shadcn/ui Library eine Tabelle erstellt.

Der Header der Tabelle beinhaltet einerseits die aus der Datenbank gelieferten Felder Id, Name, Beschreibung und Datum, andererseits die Aktionen Ansehen, Bearbeiten und Löschen, welche das CRUD von Teams ermöglichen.

Oberhalb der Tabelle befindet sich zusätzlich ein Button für das Erstellen eines neuen Teams. In der Tabelle werden die erhaltenen Daten mit einer Map Funktion reihenweise dargestellt. Die Buttons für die Aktionen wurden ebenfalls eingefügt, jedoch zunächst ohne Funktionalität.

Delete Button Funktionalität:

Für das Löschen eines Teams wurde eine neue Datei «actions.ts» kreiert. In dieser Datei wird zunächst überprüft, ob eine gültige NextAuth Session vorhanden ist. Anschliessend wird eine Anfrage mit der Methode «DELETE» an das Backend gesendet, um das entsprechende Team zu löschen. Bei erfolgreichem Löschen wird eine Erfolgsmeldung, bei einem Fehler eine Fehlermeldung zurückgegeben.

Für die Benutzeroberfläche des Delete Buttons wurden aus der shadcn/ui Library die Komponenten «AlertDialog» und «Sonner» verwendet. Durch das Klicken auf den Löschen Button erscheint ein Popup, welches den Benutzer fragt, ob das Team wirklich gelöscht werden möchte. Erst nach Bestätigung dieses Dialogs, wird das Team wirklich gelöscht. Nach erfolgreichem Löschen wird zusätzlich ein Toast angezeigt, der den Vorgang bestätigt.

Create Button Funktionalität:

Für das Erstellen eines neuen Teams wurde eine separate Seite mit einem Formular umgesetzt. Beim Klicken auf den «Neues Team erstellen» Button oberhalb der Tabelle wird der Benutzer auf diese Seite weitergeleitet.

Die Seite besteht aus zwei zentralen Komponenten. Auf der rechten Seite befindet sich die Komponente «SquadForm», welche die allgemeinen Teaminformationen wie Name, Beschreibung und Datum enthält. Auf der linken Seite befindet sich die «SquadBuilder» Komponente, welche das Zusammenstellen des Teams ermöglicht. Hier können Spieler den fünf Positionen Sturm, Mittelfeld, Verteidigung, Torwart und Ersatz hinzugefügt werden. Ausserdem können Trainer dem Team hinzugefügt werden.

Die Eingaben des Formulars werden vor dem Absenden validiert, um sicherzustellen, dass alle Felder korrekt ausgefüllt sind.

Beim Absenden des Formulars wird eine Anfrage mit der Methode «POST» an das NestJS Backend gesendet, welche das neue Team erstellt, und der Benutzer wird auf die Übersichtsseite der Teams weitergeleitet.

Edit Button Funktionalität:

Für die Bearbeitung eines bestehenden Teams wurde ebenfalls eine separate Seite mit einem Formular erstellt. Beim Klicken auf das Stift Icon in der Tabelle wird der Benutzer auf diese Seite weitergeleitet.

Zunächst werden die aktuellen Daten des ausgewählten Teams vom NestJS Backend abgefragt. Für die Darstellung der Daten werden die gleichen beiden Komponenten «SquadForm» und «SquadBuilder» verwendet. Die Daten werden auf der Seite vorausgefüllt angezeigt, sodass der Benutzer die bestehenden Informationen anpassen kann.

Nach der Bearbeitung der Daten kann der Benutzer die Änderungen speichern oder verwerfen. Beim Speichern wird eine Anfrage mit der Methode «PUT» an das Backend gesendet, welche die aktualisierten Daten übernimmt, und den Benutzer auf die Übersichtsseite der Teams weiterleitet.

Frontend Squad Bereich erstellen:

Für die Anzeige eines Teams wurde eine öffentliche Frontend Seite erstellt. Die URL wird anhand der Team ID generiert. Damit die Datenbank ID nicht ersichtlich und nicht erratbar ist, wird sie mit der Library «Hashids» verschlüsselt.

Die Teamdaten werden über die Backend API abgerufen und anschliessend grafisch dargestellt. Oben links wird der Name des Teams und das Datum angezeigt.

Die Startaufstellung des Teams wird auf der linken Seite auf einem grafischen Fussballfeld dargestellt. Die Spieler werden nach ihrer Position (Sturm, Mittelfeld, Verteidigung und Torwart) sortiert dargestellt.

Die Trainer und die Ersatzspieler werden auf der rechten Seite dargestellt, um eine klare Trennung zwischen Startspielern Ersatz und Trainer zu erzeugen.

Um das Teilen eines Teams zu erleichtern, wurde ein «Link Kopieren» Button eingebaut, welcher die URL im Clipboard abspeichert.

Backend Player Bereich erstellen:

Für die Verwaltung der Spieler wurde im Admin Bereich eine separate Seite erstellt. Auf dieser Seite werden alle vorhandenen Spieler vom NestJS Backend abgefragt und in einer Übersicht dargestellt.

Neue Spieler können über ein Formular erfasst werden. Bestehende Spieler können bearbeitet und gelöscht werden. Für das Erstellen und Bearbeiten von Spielern wird ein Sheet verwendet, das sich auf der rechten Seite öffnet.

Die Eingaben werden validiert und anschliessend über die Backend API and die Datenbank übermittelt.

Backend Trainer Bereich erstellen:

Für die Verwaltung der Trainer wurde im Admin Bereich ebenfalls eine separate Seite erstellt. Der Aufbau orientiert sich am Spieler Bereich, damit die Bedienung einheitlich ist. Alle vorhandenen Trainer werden auf dieser Seite angezeigt.

Die Eingabe erfolgt auch hier über ein sich auf der rechten Seiten öffnendes Sheet. Durch die einheitliche Struktur von Spieler und Trainer Bereich konnte die Umsetzung effizient implementiert werden und eine einheitliche Benutzerführung konnte sichergestellt werden.

Für die Applikation wurde bewusst das Meta-Tag «noindex, nofollow» gesetzt. Da es keine öffentlichen für alle zugänglichen Seiten gibt, soll verhindert werden, dass die Seiten von Suchmaschinen indexiert werden.

2.5.4 Datenbank

Die Datenbank wird vollständig vom zur Verfügung gestellten NestJS Backend verwaltet. Das Frontend greift nicht direkt auf die Datenbank zu, sondern kommuniziert ausschliesslich über die des Backends bereitgestellten API-Endpunkte.

Die Struktur der Datenbank wurde im Backend bereits definiert und beinhaltet die Tabellen Team, Spieler, Trainer und die Zwischentabellen Teamspieler und Teamtrainer.

Da die Datenbank nicht Teil der IPA ist, wurden keine Änderungen an der Datenbankstruktur vorgenommen.

2.5.5 Anbindung an die Datenbank

Die Verbindung zur Datenbank erfolgt indirekt durch das bestehende NestJS Backend. Das Frontend sendet HTTP-Anfragen an die API-Endpunkte des Backends. Das Backend verarbeitet die Anfragen und kommuniziert danach mit der Datenbank.

Für die Kommunikation wurden zwei zentrale Funktionen in der Datei «lib/nest-api.ts» verwendet. Die erste Funktion erstellt mittels eines Logins vor jeder Anfrage einen neuen Bearer Token und die zweite Funktion fügt diesen Bearer Token in den Header der Anfrage hinzu.

So kann das Frontend die notwendigen Daten abrufen, erstellen, bearbeiten oder löschen, ohne direkt mit der Datenbank zu interagieren.

2.5.6 Routen

Das Routing der Applikation wird durch den Next.js App Router umgesetzt.

Beim App Router wird jede Datei mit dem Namen «page.tsx» automatisch zu einer Route.

Alle geschützten Routen befinden sich im admin Ordner und werden durch NextAuth vor unbefugtem Zugriff geschützt.

Die folgende Abbildung zeigt einen Ausschnitt der Ordnerstruktur des Projekts:

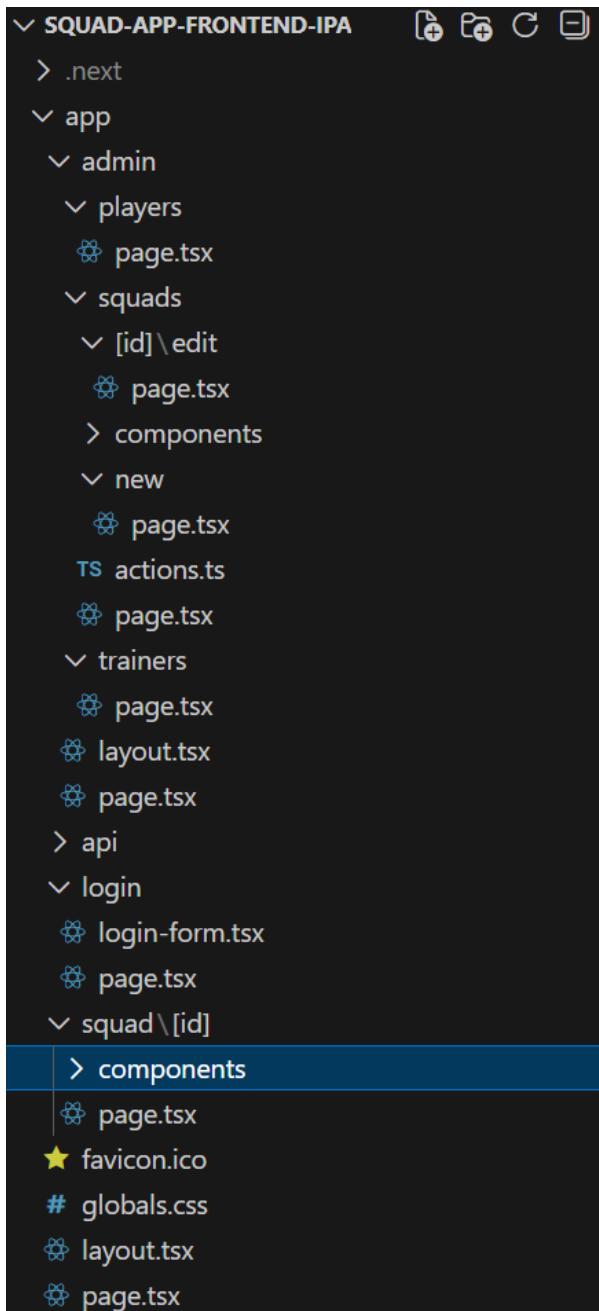


Abbildung 12: Ordnerstruktur der Next.js Routen
Quelle: Eigene Darstellung

2.5.7 Implementierung der Kernfeatures

Die wichtigsten Funktionen der Applikation bestehen aus der Verwaltung von Teams, Spielern und Trainern sowie der öffentlichen Darstellung eines Teams. Die folgenden Abschnitte zeigen die Umsetzung dieser Kernfeatures.

Team Verwaltung:

Der Team Bereich ist die Startseite des Admin Bereiches. Alle bestehenden Teams werden in einer Liste dargestellt. Für jedes Team wurden vollständige CRUD-Funktionalitäten implementiert.

Der Team Bereich wurde als erste der drei Verwaltungsseiten umgesetzt, da er die komplexeste Funktionalität enthält. Durch Klicken auf den «Bearbeiten»- oder «Neues Team erstellen»- Button öffnet sich folgende Ansicht.

Oben befindet sich ein Formular, in dem allgemeine Informationen eines Teams wie Name, Beschreibung und Datum eingegeben werden können.

Darunter befindet sich auf der linken Seite ein Fussballfeld mit den vier Positionen Sturm, Mittelfeld, Verteidigung und Torwart und auf der rechten Seite die Trainer und Ersatzspieler. Den einzelnen Positionen können Spieler oder Trainer über eine Combobox aus der shadcn/ui Library hinzugefügt oder entfernt werden.

Abbildung 13: Bearbeiten eines Teams im Admin Bereich

Quelle: Eigene Darstellung

Team Darstellung:

Neben dem Admin Bereich existiert eine öffentliche Ansicht eines Teams. Diese Seite ist für alle zugänglich und dient dazu, ein Team mit anderen Personen zu teilen.

Die Darstellung zeigt ein Fussballfeld mit den vier definierten Positionen sowie die Trainer und Ersatzspieler. Die Seite kann über einen Share-Link oder über die Aktion «Ansehen» in der Team Seite im Admin Bereich geöffnet werden.

Die Daten können in dieser Ansicht ausschliesslich gelesen, nicht jedoch bearbeitet werden.

Waldwölfe

18.9.2024



Abbildung 14: Öffentliche Team Darstellung

Quelle: Eigene Darstellung

Spieler Verwaltung:

In der Spieler Verwaltung können alle Spieler erstellt, bearbeitet und gelöscht werden. Alle Spieler werden in einer Liste dargestellt und können über entsprechende Aktionen verwaltet werden.

Beim Erstellen oder Bearbeiten öffnet sich auf der rechten Seite ein Formular in einem sogenannten «Sheet». Dieses enthält die Felder Vorname, Nachname und Geschlecht.

Diese Felder sind Pflichtfelder und werden vor dem Speichern validiert. Bei fehlerhaften Eingaben wird eine Fehlermeldung angezeigt.

Nur bestehende Spieler können einem Team hinzugefügt werden. Die Spieler Verwaltung bildet somit die Grundlage für die Team Verwaltung.

Spieler Bereich

Neuen Spieler erstellen

Vorname	Nachname	Geschlecht	Bearbeiten
Deane	Rutlandasdf	Female	
Armando	Meier	Female	
Bianca	Goose	Other	
Florian	Tison	Other	
Emeline	Morten	Other	
Haslett	Leatherland	Other	
Brook	Lamburn	Other	
Cecile	Selvey	Other	
Lucy	Sinnie	Other	
Hildegarde	Prickett	Other	

Spieler bearbeiten

Vorname: Armando

Nachname: Meier

Geschlecht: ☐ Männlich ☒ Weiblich ☐ Divers

Speichern **Abbrechen**

Abbildung 15: Bearbeiten eines Spielers im Admin Bereich

Quelle: Eigene Darstellung

Trainer Verwaltung:

Die Trainer Verwaltung wurde nach dem gleichen Prinzip wie die Spieler Verwaltung umgesetzt. Hier können Trainer erstellt, bearbeitet oder gelöscht werden.

Alle Trainer werden in einer Liste dargestellt. Auch hier öffnet sich beim Bearbeiten oder Erstellen auf der rechten Seite ein Formular.

Durch die einheitliche Struktur von Spieler und Trainer Verwaltung konnte die Benutzeroberfläche konsistent gestaltet und die Umsetzung effizient durchgeführt werden.

Trainer Bereich

Neuen Trainer erstellen

Vorname	Nachname	Geschlecht	Bearbeiten
Chadd	Czajkowski	Female	
Tuesday	Gunn	Other	
Jillie	MacCrossan	Other	
Ariana	Dutton	Other	
Waylon	Cromett	Other	
Mickey	Woodcock	Other	
Linette	Yates	Other	
Konrad	Whyffen	Other	
Osbourne	Cheng	Other	
Allyson	Sapauton	Other	

Trainer bearbeiten

Vorname: Ariana

Nachname: Dutton

Geschlecht: ☐ Männlich ☒ Weiblich ☐ Divers

Speichern **Abbrechen**

Abbildung 16: Bearbeiten eines Trainers im Admin Bereich

Quelle: Eigene Darstellung

Die Umsetzung dieser Kernfeatures bildet die Grundlage der gesamten Applikation und ermöglicht eine einfache Verwaltung sowie Darstellung von Fussballteams.

2.5.8 Vorbereitungen für den Upload / Live-Schaltung / Implementierung

Die Live-Schaltung ist kein Teil des Projektes. Für eine produktive Inbetriebnahme müssten die Environment Variablen angepasst, das Nest.js Backend auf einem Server bereitgestellt und das Frontend gebaut und deployed werden. Dies mittels einer Plattform wie Vercel oder ähnlichem.

2.6 Kontrollieren

In diesem Kapitel werden die Bedingungen der Testumgebung beschrieben sowie die in der Planungsphase definierten Tests durchgeführt.

2.6.1 Beschreibung der Randbedingungen / Testanlage (Umfeld)

Die Tests wurden auf dem Entwicklungsrechner des Autors durchgeführt.

Folgende Spezifikationen wurden für das Testen verwendet:

Software:

- Windows 11 Pro 26200.7623
- Next.js v16.1.6
- Node.js v20
- NestJS Backend (bestehendes Projekt)
- Google Chrome v144
- Laragon für Datenbank

Hardware:

- CPU: Intel i7-12700H
- GPU: RTX 3060 Laptop Version
- RAM: 16 GB

Folgende Voraussetzungen sind für das Testen notwendig:

1. Repository klonen und Dependencies installieren mit: `npm install`
2. Datenbank über Laragon starten
3. NestJS Backend in `.env` sicherstellen, dass der Port frei ist, z. Bsp. `PORT=3001`. Backend starten (`npm run start`)
4. Frontend `.env` konfigurieren und starten (`npm run dev`).

5. Applikation im Browser öffnen (<http://localhost:3000>)
6. Folgendes Login eingeben: Name: admin Passwort: admin

2.6.2 Testprotokoll

Alle in der Planen-Phase definierten Testfälle werden jetzt hier durchgegangen

Die Tests wurden ausschliesslich manuell durchgeführt. Obwohl automatisierte Tests mit Tools wie Playwright möglich wären, würde deren Implementierung in der gegebenen Zeit einen zu hohen Aufwand darstellen.

Da das Projekt eine grafische Benutzeroberfläche enthält, lassen sich die definierten Tests effizient durch manuelle GUI-Tests im Browser überprüfen

#	1
Testfall	Login erfolgreich
Vorbedingung	Admin Passwort und Name vorhanden
Getestet von	Flurin Bandli
Getestet am	12.03.2026
Eingabe	Login Seite (/login) öffnen, gültigen Benutzernamen und Passwort eingeben und auf «Login» klicken.
Erwartetes Ergebnis	Der Benutzer wird in den geschützten Admin Bereich (/admin/squads) weitergeleitet.
Effektives Ergebnis	Eine Weiterleitung auf /admin/squads erfolgt.
i.O.	Ja
Fazit	Das Login funktioniert wie erwartet.

#	2
Testfall	Login fehlgeschlagen
Vorbedingung	Keine
Getestet von	Flurin Bandli
Getestet am	12.03.2026
Eingabe	Login Seite (/login) öffnen und ein falsches Passwort eingeben.
Erwartetes Ergebnis	Der Benutzer bleibt auf der Login Seite und eine Fehlermeldung wird angezeigt
Effektives Ergebnis	Eine Fehlermeldung erscheint und es erfolgt keine Weiterleitung.
i.O.	Ja
Fazit	Ungültige Login Versuche werden korrekt erkannt und dem Benutzer mitgeteilt.

Admin Login

Ungültige Anmeldedaten

Benutzername

admin

Passwort

....

→] **Login**

Abbildung 17: Fehlermeldung bei ungültigem Login

Quelle: Eigene Darstellung

#	3
Testfall	Unbefugter Zugriff auf geschützten Bereich
Vorbedingung	Benutzer ist ausgeloggt
Getestet von	Flurin Bandli
Getestet am	12.03.2026
Eingabe	Direkt die URL /admin/squads im Browser öffnen, ohne eingeloggt zu sein.
Erwartetes Ergebnis	Der Benutzer wird automatisch auf die Login Seite weitergeleitet.
Effektives Ergebnis	Weiterleitung auf /login erfolgt.
i.O.	Ja
Fazit	Der geschützte Bereich ist korrekt abgesichert.

#	4
Testfall	Logout
Vorbedingung	Benutzer ist eingeloggt
Getestet von	Flurin Bandli
Getestet am	12.03.2026
Eingabe	Im Admin Bereich auf den «Logout» Button klicken.
Erwartetes Ergebnis	Der Benutzer wird ausgeloggt und automatisch auf die Login Seite weitergeleitet.

Effektives Ergebnis	Der Benutzer wurde ausgeloggt und auf die Login Seite weitergeleitet.
i.O.	Ja
Fazit	Das Logout funktioniert wie erwartet und beendet die Session des Benutzers.

#	5
Testfall	Trainer hinzufügen
Vorbedingung	Benutzer ist eingeloggt
Getestet von	Flurin Bandli
Getestet am	12.03.2026
Eingabe	/admin/trainers öffnen, auf «Neuen Trainer erstellen» klicken, Vorname, Nachname und Geschlecht eingeben und speichern.
Erwartetes Ergebnis	Der neue Trainer wird erstellt und erscheint in der Liste der Trainer.
Effektives Ergebnis	Der Trainer wurde erfolgreich erstellt und in der Liste angezeigt.
i.O.	Ja
Fazit	Trainer können erfolgreich hinzugefügt werden.

#	6
Testfall	Spieler löschen
Vorbedingung	Benutzer ist eingeloggt
Getestet von	Flurin Bandli
Getestet am	12.03.2026
Eingabe	/admin/players öffnen, bei einem bestehenden Spieler das Löschen Symbol klicken und das Löschen bestätigen.
Erwartetes Ergebnis	Spieler wird gelöscht und aus der Liste entfernt. Ein Toast zeigt das erfolgreiche Löschen an.
Effektives Ergebnis	Spieler wurde gelöscht und aus der Liste entfernt. Ein Toast zeigt das erfolgreiche Löschen an.
i.O.	Ja
Fazit	Spieler können nach Bestätigung gelöscht werden.

#	7
Testfall	Team bearbeiten
Vorbedingung	Team existiert und ist im Admin Squad Bereich sichtbar
Getestet von	Flurin Bandli
Getestet am	12.03.2026
Eingabe	/admin/squads öffnen, bei einem bestehenden Team das Bearbeiten Symbol klicken, Änderungen vornehmen und speichern.
Erwartetes Ergebnis	Die Änderungen werden gespeichert und sowohl in der Teamübersicht als auch in der öffentlichen Sicht angezeigt.
Effektives Ergebnis	Die Änderungen wurden erfolgreich gespeichert und werden korrekt dargestellt.
i.O.	Ja
Fazit	Teams können erfolgreich bearbeitet werden.

Team Infos

Teamname

Die Feuerfuchse

Beschreibung

Eine aggressive Offensivmannschaft mit schnellen Flügelspielern.

Datum

09/09/2024

Team aktualisieren

Abbrechen

Aufstellung

Sturm

Haslett Leatherland
✕

Mittelfeld

Bianca Goose
✕

Grantham Trever
✕

Florian Tison
✕

Cicely Bearward
✕

Emeline Morten
✕

Jessamyn Wiffen
✕

Verteidigung

Rosina Darkott
✕

Armando Meier
✕

Torwart

Trainer

Tuesday Gumn
✕

Jillie MacCrossan
✕

Karlis Stroud
✕

Ersatz

Kristyn Gosenell
✕

Menard Lymbourne
✕

Rici Currum
✕

Abbildung 18: Bearbeiten eines Spielkaders im Admin Bereich.
Quelle: Eigene Darstellung

#	8
Testfall	Share Link funktioniert
Vorbedingung	Benutzer ist eingeloggt, im Admin Squads Bereich existiert ein Squad.
Getestet von	Flurin Bandli
Getestet am	12.03.2026
Eingabe	Bei einem Team auf das Ansehen Symbol klicken. Im neu geöffneten Tab, der die öffentliche Teamansicht zeigt, auf «Link kopieren» klicken.
Erwartetes Ergebnis	Der Link der URL wird im Clipboard abgespeichert und kann im Browser eingefügt werden.
Effektives Ergebnis	Der Link der URL wurde im Clipboard abgespeichert.
i.O.	Ja
Fazit	Der Share Link funktioniert und ein Team kann mühelos geteilt werden.

9.9.2024

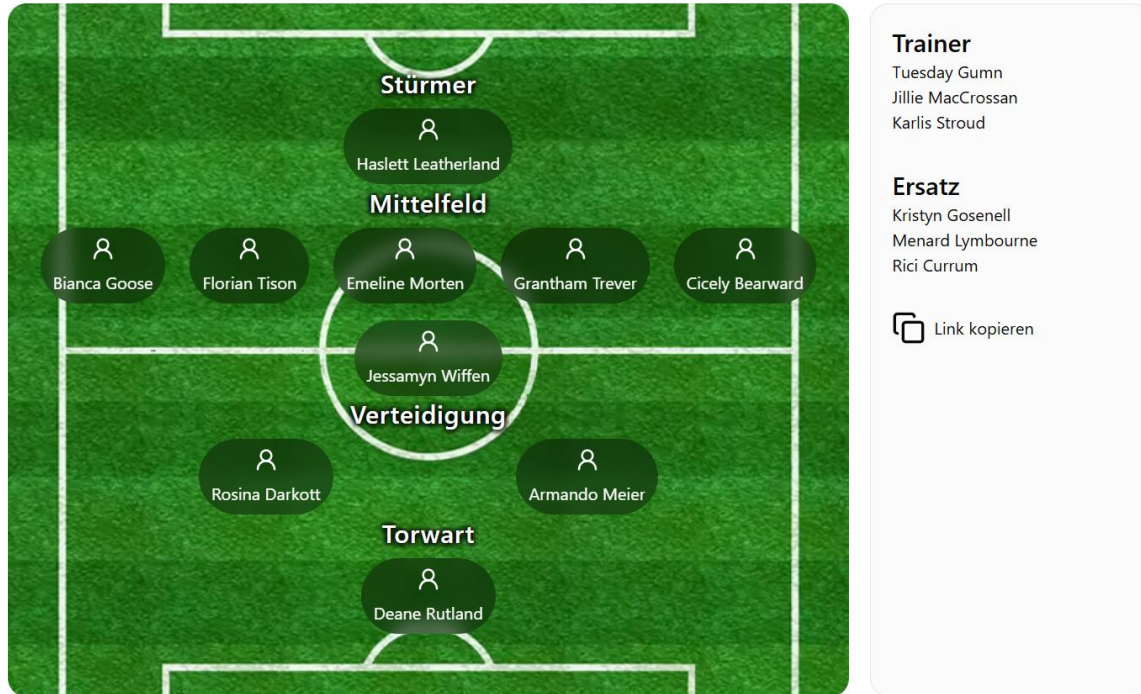


Abbildung 19: Kopieren des öffentlichen Links eines Spielkaders.
Quelle: Eigene Darstellung

#	9
Testfall	Pflichtfelder Validierung
Vorbedingung	Benutzer ist eingeloggt
Getestet von	Flurin Bandli
Getestet am	12.03.2026
Eingabe	Im Formular zum Erstellen oder Bearbeiten eines Spielers oder Trainers ein Feld leer lassen und versuchen das Formular zu speichern.
Erwartetes Ergebnis	Das Formular wird nicht gespeichert und beim leeren Feld erscheint eine entsprechende Fehlermeldung.
Effektives Ergebnis	Das Formular wurde nicht gespeichert und eine entsprechende Fehlermeldung wurde angezeigt.
i.O.	Ja
Fazit	Die Validierung der Pflichtfelder funktioniert korrekt.

Spieler erstellen

×

Vorname

Vorname ist erforderlich

Nachname

Geschlecht

☒ Männlich ☐ Weiblich ☐ Divers

Speichern

Abbrechen

Abbildung 20: Validierung von Pflichtfeldern im Formular

Quelle: Eigene Darstellung

#	10
Testfall	Gelöschter Spieler in bestehendem Team
Vorbedingung	Team existiert und enthält mindestens einen Spieler
Getestet von	Flurin Bandli
Getestet am	12.03.2026
Eingabe	Einen Spieler löschen, der bereits einem bestehenden Team zugeordnet ist. Anschliessend die öffentliche Teamansicht des Teams öffnen.
Erwartetes Ergebnis	Der gelöschte Spieler wird im Team nicht mehr angezeigt. Die restliche Darstellung des Teams funktioniert weiterhin korrekt.
Effektives Ergebnis	Der gelöschte Spieler wurde nicht mehr angezeigt und die Teamdarstellung funktionierte weiterhin ohne Fehler.
i.O.	Ja
Fazit	Die Applikation verarbeitet gelöschte Spieler in bestehenden Teams korrekt.

Teams

Spieler

Trainer

Logout

Teams Bereich

Neues Team erstellen

Id	Name	Beschreibung	Datum	Ansehen	Bearbeiten	Löschen
1	Die Feuerfuchse	Eine aggressive Offensivmannschaft mit schnellen Flügelspielern.	9.9.2024			
2	Sturmkrähend	Nicht bekannt für ihre unberechenbaren Konter und hohe Pressingintensität.	4.8.2024			
3	Eisadler	Disziplinierte Defensivformation, die auf Standardsituationen spezialisiert ist.	4.10.2024			
4	Waldwölfe	Körperbetontes Spiel durch kraftvolle Mittelfeldspieler.	18.9.2024			
5	Nebelpanther	Technisch versierte Mannschaft mit starkem Besitzspiel.	30.8.2024			
6	Donnerbären	Physikalisch starkes Team, das auf Zweikämpfe setzt.	12.10.2024			
7	Sonnenfalken	Offensiv ausgerichtete Truppe mit kreativem Offensivspiel.	25.9.2024			
8	Mondwölfe	Taktisch variabel, oft in defensiver Grundordnung.	15.8.2024			
9	Gletscherfuchse	Geduldiges Aufbauspiel und präzise Flanken.	8.10.2024			

Abbildung 21: Admin Bereich. Übersicht der Teams

Quelle: Eigene Darstellung

#	11
Testfall	Gelöschter Trainer in bestehendem Team
Vorbedingung	Team existiert und enthält mindestens einen Trainer
Getestet von	Flurin Bandli
Getestet am	12.03.2026
Eingabe	Einen Trainer löschen, der bereits einem bestehenden Team zugeordnet ist. Anschliessend die öffentliche Teamansicht des Teams öffnen.
Erwartetes Ergebnis	Der gelöschte Trainer wird im Team nicht mehr angezeigt. Die restliche Darstellung des Teams funktioniert weiterhin korrekt.
Effektives Ergebnis	Der gelöschte Trainer wurde nicht mehr angezeigt und die Teamdarstellung funktionierte weiterhin ohne Fehler.
i.O.	Ja
Fazit	Die Applikation verarbeitet gelöschte Trainer in bestehenden Teams korrekt.

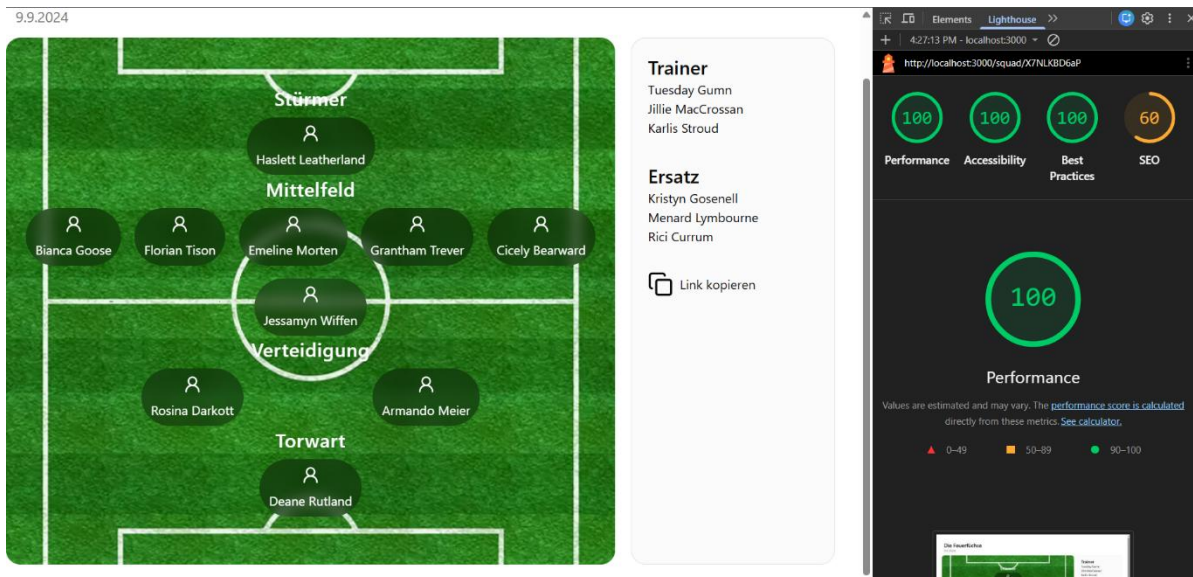


Abbildung 22: Lighthouse Score

Quelle: Eigene Darstellung

#	12
Testfall	Lighthouse Performance Test
Vorbedingung	Applikation läuft lokal
Getestet von	Flurin Bandli
Getestet am	12.03.2026
Eingabe	Seite in Google Chrome öffnen, F12 Taste drücken, Lighthouse starten.
Erwartetes Ergebnis	Performance Score > 90%, Accessibility > 90%, Best Practices > 90%, SEO kann tiefer sein
Effektives Ergebnis	Performance, Accessibility und Best Practices 100%. SEO 60%
i.O.	Ja
Fazit	Das Ergebnis ist perfekt. Die Applikation ist also top.

2.6.3 Bugfixing

Während der Durchführung der Tests konnten keine kritischen Fehler festgestellt werden. Alle definierten Testfälle konnten erfolgreich durchgeführt werden.

Im Rahmen der Tests wurden jedoch kleinere Verbesserungen im Benutzerinterface vorgenommen, insbesondere im Admin Squad Bereich.

2.7 Auswerten

In diesem Kapitel wird das Projekt kritisch reflektiert. Das Ziel ist es, aus Erfahrungen zu lernen, zu analysieren wo und wieso Probleme aufgetreten sind und wie diese in Zukunft zu handhaben sind.

2.7.1 Reflexion der Vorgehensweise

Die gewählte Projektmanagementmethode IPERKA hat sich für dieses Projekt als sehr sinnvoll erwiesen. Die klare Struktur der einzelnen Phasen half bei der Umsetzung des Projekts enorm. Schritt für Schritt konnte das Projekt geplant, umgesetzt und dokumentiert werden.

In der Phase Informieren wurden zunächst die Anforderungen des Projekts analysiert und die vorhandenen Systeme angeschaut. Das Verständnis des bestehenden NestJS Backends war dabei besonders von Bedeutung, da das Frontend ausschliesslich über dessen API mit der Datenbank kommuniziert.

In der Planungsphase wurden ein Use-Case Diagramm, ein Route Map Diagramm sowie das Testkonzept erstellt. Diese Planung half dabei, während der Umsetzung stets den Überblick über die nächsten Schritte zu behalten.

In der Realisierungsphase wurde die Applikation Schritt für Schritt umgesetzt. Dabei wurde zuerst die grundlegende Struktur der Applikation erstellt, anschliessend die Backend Anbindung implementiert und danach die einzelnen Funktionen wie das Verwalten von Teams, Spielern und Trainern und die Darstellung von Teams umgesetzt.

Rückblickend bin ich mit der gewählten Reihenfolge der Arbeitsschritte äusserst zufrieden. Besonders hilfreich war es, zuerst die grundlegenden technischen Komponenten wie die Authentifizierung und die Backend Kommunikation umzusetzen, bevor die eigentlichen Funktionen entwickelt wurden.

2.7.2 Bewertung des Produktes

Über einen geschützten Admin Bereich können Teams, Spieler und Trainer erstellt, bearbeitet und gelöscht werden. Teams können über einen öffentlichen Link aufgerufen, weitergeleitet und grafisch dargestellt werden.

Alle im Projektauftrag definierten Kernfunktionen wurden erfolgreich umgesetzt und getestet. Die Benutzeroberfläche ist übersichtlich aufgebaut und ermöglicht eine einfache Bedienung. Besonders zufrieden bin ich mit der Benutzeroberfläche des Admin Bereichs. Er ist meiner Meinung nach sehr einfach zu bedienen und schützt den Benutzer dank Validierung, Tooltips auf den Icons und einer Löschbestätigung vor Fehlern oder Missgeschicken.

Auch die klare Trennung zwischen Frontend und Backend über definierte API-Endpunkte hat sich als sehr vorteilhaft erwiesen. Dadurch blieb die Struktur der Applikation übersichtlich und Änderungen konnten einfacher umgesetzt werden.

Im Verlauf des Projekts gab es immer wieder Ideen für Erweiterungen oder Verbesserungen, die ich aufgrund des Zeitdrucks nach hinten verschoben hatte. Gewisse Erweiterungen konnten eingebaut werden, andere leider nicht.

Die grössten Erweiterungen, die ich gerne noch implementiert hätte, sind:

1. In der Darstellung des Teams hätte ich gerne mit einer Animation Library wie «framer-motion» gearbeitet. Die Spieler hätten dabei schrittweise auf dem Spielfeld erscheinen können, ähnlich wie bei der Aufstellung vor einem Fussballspiel.
2. Im Admin Bereich hätte ich gerne eine Pagination und ein Suchfeld eingebaut. Da shadcn/ui beide dieser Komponenten hat, hätte dies auch hervorragend ins Design der Seiten gepasst und der Aufwand wäre nicht allzu gross dafür gewesen.
3. Die Anzahl Spieler pro Position und die Anzahl aktiver Spieler ist momentan unbegrenzt. Eine Limite von 10 Feldspielern und einem Torwart gäbe dem Projekt klarere Grenzen für die Darstellung.

2.7.3 Abweichungen zum Zeitplan

Vor allem bei der Implementierung des Squad Bereichs gab ich mir zu wenig Zeit. Ich wusste zwar im Vorhinein bereits, dass dieser Teil der komplexeste Teil meines Projekts wird, aber habe mich selbst wohl etwas überschätzt und die Komplexität unterschätzt.

Ich wusste wie wichtig und wieviel Zeit die Dokumentation ungefähr in Anspruch nimmt und bin mit meiner Schätzung dabei zufrieden.

Meine Verzögerungen konnte jedoch teilweise durch schneller erledigte Aufgaben wie das Erstellen der Diagramme ausgeglichen werden. Insgesamt blieb der Fortschritt des Projekts immer nahe am ursprünglichen Zeitplan.

Grössere Abweichungen, welche den Abschluss des Projekts verhindert hätten, gab es nicht.

2.7.4 Persönliches Schlusswort und Bilanz

Die Umsetzung dieser IPA war für mich auf der einen Seite eine stressige, auf der anderen Seite eine sehr spannende und lehrreiche Erfahrung.

Während der Arbeit konnte ich meine Kenntnisse im Bereich moderner Webentwicklung mit Next.js, React und TypeScript weiter vertiefen. Besonders lehrreich war für mich die Zusammenarbeit zwischen meinem Next.js Frontend und dem bestehenden NestJS Backend.

Die IPERKA-Methode half mir ausserordentlich bei einer strukturierten Arbeitsweise und dabei, das Projekt klar zu planen und umzusetzen.

Als wichtigste Erkenntnis für die Zukunft nehme ich mit, wie wichtig eine gute Planung und eine kontinuierliche Dokumentation während der Entwicklung sind.

Rückblickend bin ich mit meiner geleisteten Arbeit und mit dem entstandenen Ergebnis sehr zufrieden.

2.8 Glossar

In diesem Kapitel werden wichtige Fachbegriffe erklärt, welche im Verlauf der Dokumentation verwendet wurden.

A

Begriff / Abkürzung	Erklärung
API	Eine Schnittstelle, über welche zwei Programme miteinander kommunizieren können.

B

Begriff / Abkürzung	Erklärung
Backend	Der serverseitige Teil einer Applikation, der Daten verarbeitet und bereitstellt.

C

Begriff / Abkürzung	Erklärung
Combobox	Ein Eingabeelement, welches eine Liste von auswählbaren Optionen mit einer Suchfunktion kombiniert.
CRUD	Abkürzung für Create, Read, Update und Delete. Beschreibt die grundlegenden Operationen zum Verwalten von Daten.

H

Begriff / Abkürzung	Erklärung
Hashids	Eine Library zur Umwandlung von numerischen IDs in nicht erratbare Zeichenketten.

J

Begriff / Abkürzung	Erklärung
JWT (JSON Web Token)	Ein Token, das zur Authentifizierung zwischen Client und Server verwendet wird.

L

Begriff / Abkürzung	Erklärung
Lighthouse Score	Bewertung einer Website durch das Analysewerkzeug Google Lighthouse in Kategorien wie Performance, Accessibility und Best Practices.

N

Begriff / Abkürzung	Erklärung
NestJS	Ein Framework für Node.js zur Entwicklung von Backend-Applikationen.
NestJS Backend	Serverseitige Applikation, welche die API bereitstellt, und den Zugriff auf die Datenbank übernimmt.
Next.js	Ein Framework basierend auf React zur Entwicklung moderner Webapplikationen.
NextAuth	Eine Library zur Implementierung von Authentifizierung in Next.js Applikationen.

S

Begriff / Abkürzung	Erklärung
SEO (Search Engine Optimization)	Massnahmen zur Optimierung von Webseiten für Suchmaschinen.
shadcn/ui	Eine UI-Komponentenbibliothek für React, die besonders gut mit Tailwind CSS integriert ist.

T

Begriff / Abkürzung	Erklärung
Tailwind CSS	Ein CSS-Framework, das mit vordefinierten Utility-Klassen arbeitet.
Toast	Kurze visuelle Benachrichtigung in der Benutzeroberfläche, die eine Aktion bestätigt oder eine Fehlermeldung anzeigt.
Token	Eine Zeichenkette, die zur Authentifizierung bei Anfragen an ein Backend verwendet wird.

2.9 Quellenverzeichnis

In diesem Kapitel werden alle Quellen aufgeführt, welche während der Umsetzung des Projekts sowie beim Erstellen der Dokumentation konsultiert wurden.

NextAuth Dokumentation

<https://authjs.dev/getting-started/installation?framework=Next.js>

shadcn/ui Dokumentation

<https://ui.shadcn.com>

Material UI Dokumentation

<https://mui.com/material-ui/>

Ant Design Dokumentation

<https://ant.design/components/overview>

Hashids npm Package

<https://www.npmjs.com/package/hashids>

GitLab Repository Mirroring

<https://docs.gitlab.com/user/project/repository/mirror/>

Next.js Dokumentation

<https://nextjs.org/docs>

React Dokumentation

<https://react.dev/>

Tailwind CSS Dokumentation

<https://tailwindcss.com/docs>

3. Anhang

3.1 Git-Commit-History

Während der gesamten Entwicklungsphase wurde der Code regelmässig in das firmeninterne GitLab Repository gepusht.

Dabei wurden sinnvolle Commit Messages verwendet, um die einzelnen Entwicklungsschritte nachvollziehbar zu dokumentieren.

Die folgenden Abbildungen zeigen die Commit History des Projekts.

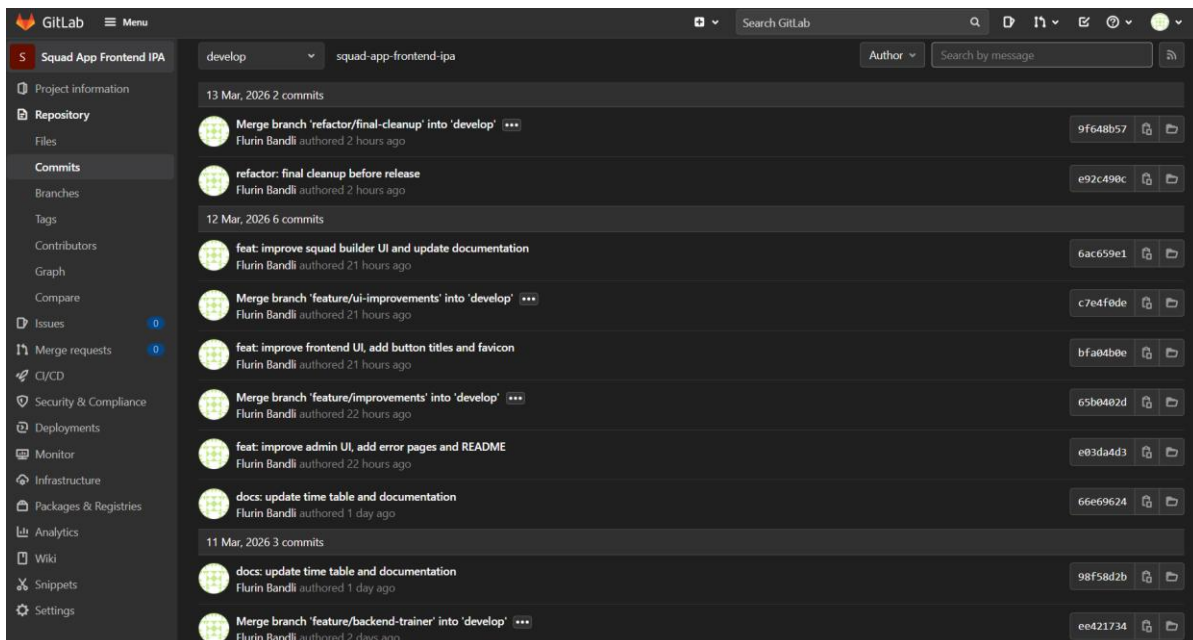


Abbildung 23: Ausschnitt der Git Commit History (Teil 1)

Quelle: Eigene Darstellung (GitLab Repository)

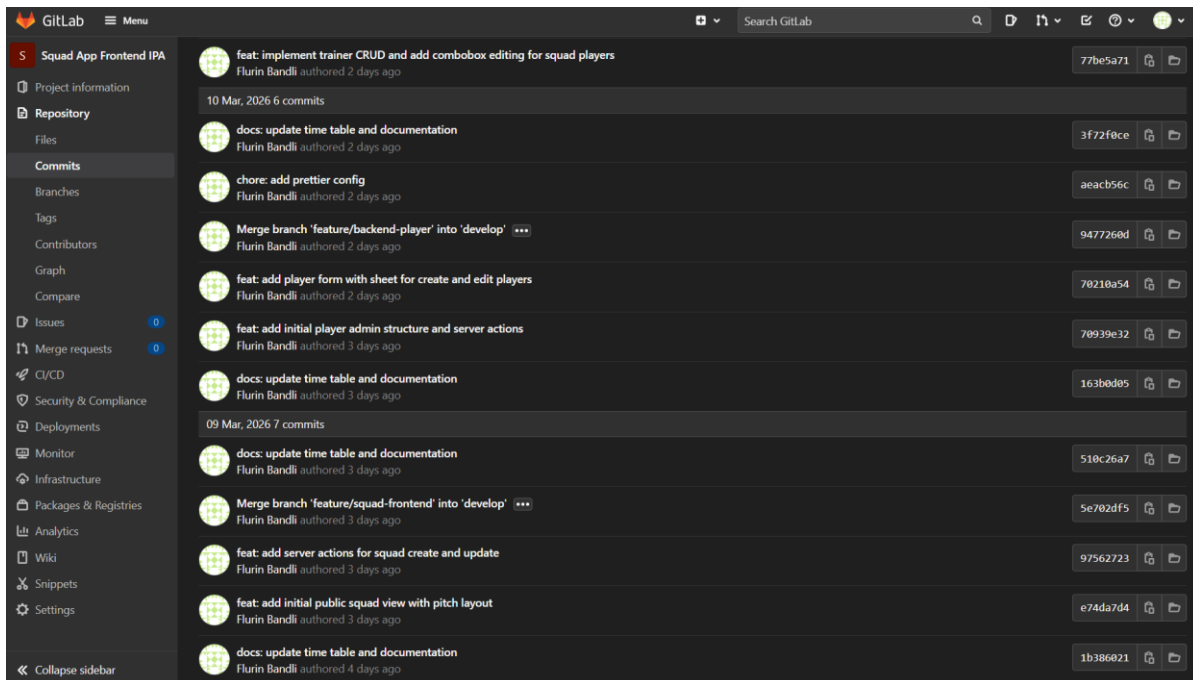


Abbildung 24: Ausschnitt der Git Commit History (Teil 2)

Quelle: Eigene Darstellung (GitLab Repository)

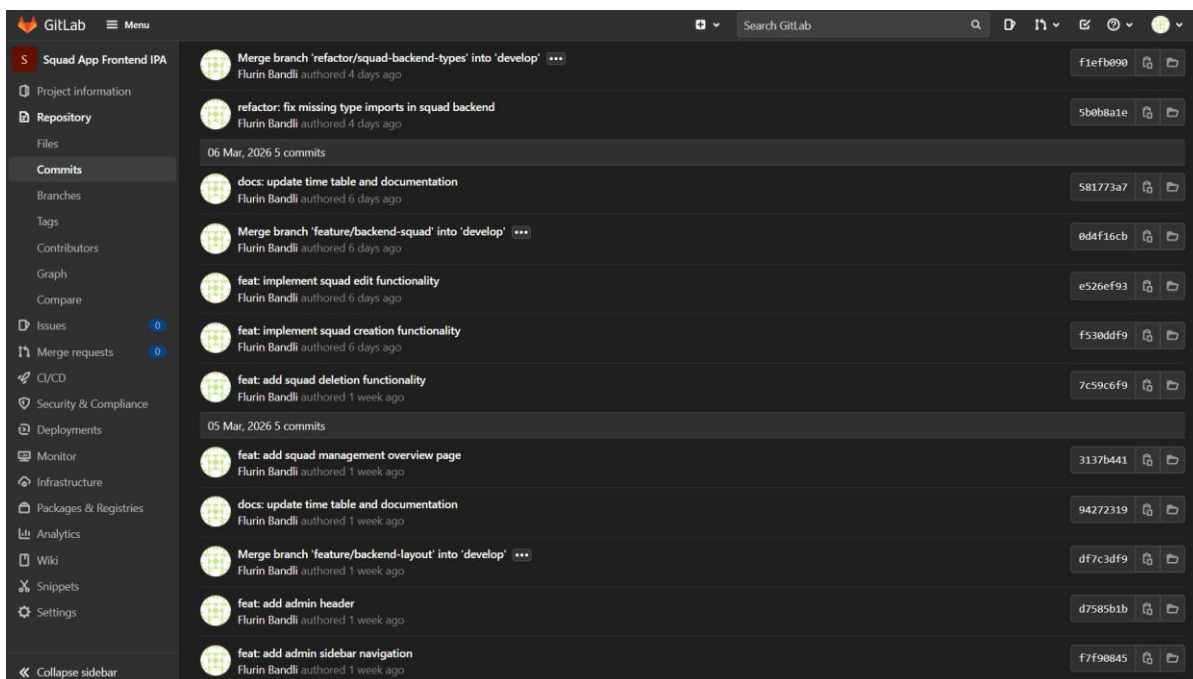


Abbildung 25: Ausschnitt der Git Commit History (Teil 3)

Quelle: Eigene Darstellung (GitLab Repository)

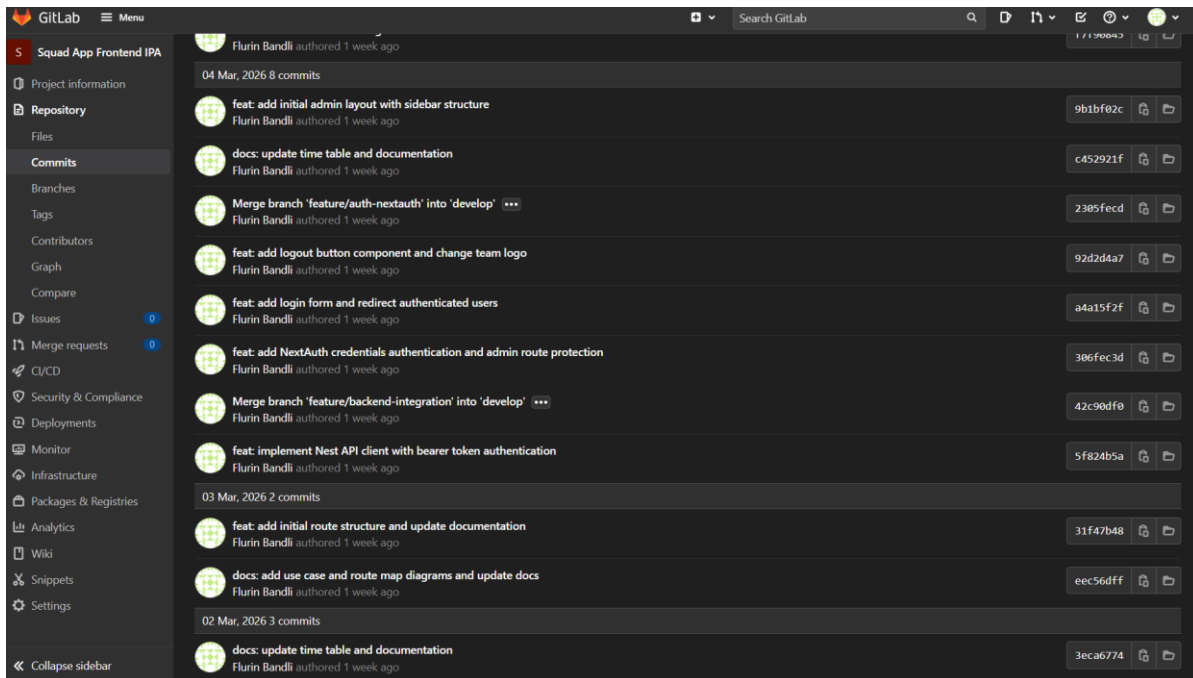


Abbildung 26: Ausschnitt der Git Commit History (Teil 4)

Quelle: Eigene Darstellung (GitLab Repository)

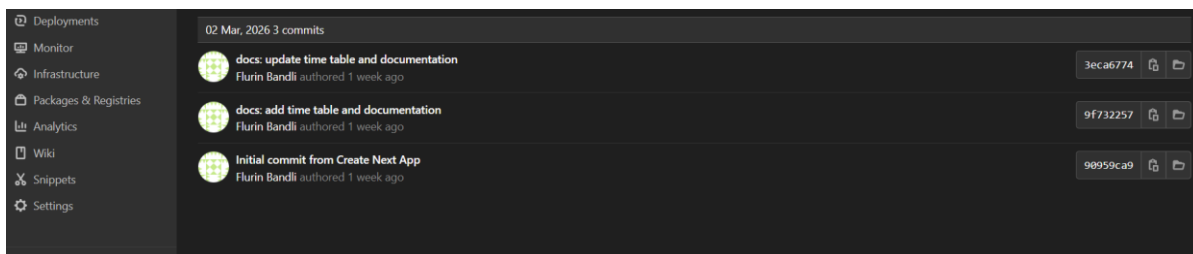


Abbildung 27: Ausschnitt der Git Commit History (Teil 5)

Quelle: Eigene Darstellung (GitLab Repository)

3.2 Projektjournal

Gesprächsprotokoll

Teilnehmer: Flurin Bandli, René Bach

Datum: 12.03.2026

Besprechungsnotizen:

- Verbesserung der Benutzeroberfläche im Admin Bereich.
- Mehr Abstand zwischen einzelnen UI-Elementen.
- Sicherstellen, dass sich Comboboxen nicht überlappen.
- Ergänzen von Icons bei Buttons zur besseren Verständlichkeit.
- Ergänzen von Tooltip Titeln bei kleinen Icons.

Getroffene Entscheidungen:

Im Erstellen / Bearbeiten vom Squad Bereich wird das Formular über der Aufstellung platziert, damit Aufstellung mehr Platz hat.

Jedem kleinen Icon einen «title» hinzufügen, damit bei Hover Text angezeigt wird.

Bei der Combobox «flex-wrap» hinzugefügt, damit sie nie überlappen.

3.3 Erster Expertenbesuch

Teilnehmer: Marius Schumacher, Flurin Bandli, René Bach

Datum, Zeit, Ort: 03.03.2026, 10:00 Uhr, Online

Besprechungsnotizen:

Glossar alphabetisch geordnet.

Keine IT eindeutigen Begriffe einbauen.

Bewertungskriterien ausdrucken und anschauen.

Legende des Zeitplans unten verschieben.

Bei Krankheit sofort Arztzeugnis einfordern und dann René und Marius informieren und auf PKorg hochladen.

Getroffene Entscheidungen:

Keine

3.4 Zweiter Expertenbesuch

Teilnehmer: Marius Schumacher, Flurin Bandli, René Bach
Datum, Zeit, Ort: 11.03.2026, 13:00 Uhr, Thurgauerstrasse 54, 8050 Zürich

Besprechungsnotizen:

Dokumentation ist das Wichtigste
Alle Kriterien aus dem Kriterienkatalog genau anschauen und überprüfen.

Dokumentation als PDF hochladen.
Externe Links sollten funktionieren und das Inhaltsverzeichnis ebenfalls.

Fokus nur auf Kriterien, nichts extra

Glossar alphabetisch geordnet
Firmenspezifische Sachen
Einzeiler pro Bild

Pro Testfall immer eigenes Fazit ziehen und Fehlerbehebung aufschreiben.

Präsentation;
Auf hochdeutsch nicht Schweizerdeutsch.
Rest auf schweizerdeutsch.
Zeit bei Testpräsentation etwas länger sein. Zeitrahmen für IPA-Präsentation 15-20 Minuten, sonst Punktabzug.
Fachgespräch ungefähr 20 Fragen.

Total 1 Stunde.

Vorgeschlagene Termine für Präsentation:
23.03.26 10:30 Uhr
24.03.26 10:30 Uhr

Getroffene Entscheidungen:
Nur noch auf das Wichtigste fokussieren. Das heisst zum einen die Dokumentation sauber abschliessen und zum anderen Alle Kriterien des Kriterienkataloges genau überprüfen und sicherstellen, dass alles Kriterien eingehalten wurden.

3.5 Code

Der vollständige Quellcode der Applikation wird als ZIP-Datei über PKorg abgegeben.

Die folgenden Codeausschnitte zeigen zentrale Teile der Implementierung.

3.5.1 lib/nest-api.ts

Diese Datei enthält zentrale Funktionen zur Kommunikation mit dem bestehenden NestJS Backend. Sie übernimmt das Login beim Backend und stellt sicher, dass bei jeder Anfrage ein Bearer Token mitgesendet wird.

```
/**
 * Central functions for interacting with the Nest API.
 * This file handles authentication and automatically attaches the access token to requests.
 */

/**
 * Authenticates with the NestJS backend and retrieves a JWT bearer token.
 * The token is required to access protected API endpoints.
 * @returns A promise that resolves to the JWT access token as a string.
 * @throws An error if authentication fails.
 */
export async function GetNestToken(): Promise<string> {
  try {
    const response = await fetch(`${process.env.NEST_API_URL}/api/auth/login`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      // Login credentials are read from environment variables for security reasons
      body: JSON.stringify({
        username: process.env.NEST_USERNAME,
        password: process.env.NEST_PASSWORD,
      }),
    });

    // Throw an error if authentication fails
    if (!response.ok) {
      throw new Error("Error authenticating with the Nest API");
    }
  }
}
```

```

    }

    // Backend response contains the JWT access token
    const data = await response.json();

    // Return the access token so it can be used for API requests
    return data.accessToken as string;
  } catch (error) {
    console.error("Error authenticating with the Nest API:", error);
    throw error;
  }
}

/**
 * Sends a request to the NestJS backend API.
 * Before making the request, it retrieves a valid JWT token using GetNestToken and includes it
 * in the Authorization header.
 * This allows you to call any protected API endpoint without worrying about authentication
 * details in your components.
 * @param endpoint The specific API endpoint to call (e.g., "api/squad").
 * @param init Optional RequestInit object to customize the fetch request (e.g., method, body).
 * @returns A promise that resolves to the response data from the API, typed as T.
 * @throws An error if the request fails or if authentication fails.
 */
export async function NestFetch<T>(
  endpoint: string,
  init: RequestInit = {}
): Promise<T> {
  try {
    // Get a valid bearer token for authentication
    const token = await GetNestToken();

    const response = await fetch(`${process.env.NEST_API_URL}${endpoint}`, {
      ...init,
      headers: {
        "Content-Type": "application/json",
        // Attach the bearer token to authorize the request
        Authorization: `Bearer ${token}`,
        ...(init.headers ?? {}),
      },
    });

    // Throw an error if the request fails
    if (!response.ok) {
      throw new Error(`Response error: ${response.statusText}`);
    }
  }
}

```

```

}

const text = await response.text();

// If the response is empty, return undefined (e.g., for DELETE requests)
if (!text) {
  return undefined as T;
}

// Return the JSON response from the API
return JSON.parse(text) as T;
} catch (error) {
  console.error(
    `Response error: ${error instanceof Error ? error.message : String(error)}`
  );
  throw error;
}
}

```

3.5.2 login-form.tsx

Diese Komponente implementiert das Login Formular der Applikation. Sie verwendet React Hook Form zur Verwaltung der Formulardaten und Zod zur Validierung der Benutzereingaben.

```

/**
 * Zod schema for validating the login form inputs.
 * It ensures that both username and password are provided.
 */

const formSchema = z.object({
  username: z.string().min(1, "Benutzername ist erforderlich"),
  password: z.string().min(1, "Passwort ist erforderlich"),
});

export default function LoginForm() {
  const router = useRouter();
  const [loginError, setLoginError] = useState<string | null>(null);

  /**
   * Initializes React Hook Form with Zod validation.
   * Default values are empty for both input fields.
   */

```

```
const form = useForm<z.infer<typeof formSchema>>({
  resolver: zodResolver(formSchema),
  defaultValues: {
    username: "",
    password: "",
  },
});

/**
 * Handles form submission.
 * Calls NextAuth's credentials provider to authenticate the user.
 */
async function onSubmit(data: z.infer<typeof formSchema>) {
  setLoginError(null);
  const result = await signIn("credentials", {
    username: data.username,
    password: data.password,
    redirect: false,
  });

  // Show error message if credentials are invalid
  if (result?.error) {
    setLoginError("Ungültige Anmeldedaten");
    return;
  }

  // Redirect authenticated users to the admin dashboard
  if (result?.ok) {
    router.push("/admin/squads");
    return;
  }
}
```

3.5.3 squad-builder.tsx

Diese Komponente ist für das Zusammenstellen einer Aufstellung zuständig.
Spieler können verschiedenen Position zugewiesen und Trainer dem Team hinzugefügt werden.

```
const [addingStriker, setAddingStriker] = useState(false);
const [addingMidfielder, setAddingMidfielder] = useState(false);
const [addingDefender, setAddingDefender] = useState(false);
```



```
const [addingGoalkeeper, setAddingGoalkeeper] = useState(false);
const [addingBackup, setAddingBackup] = useState(false);
const [addingTrainer, setAddingTrainer] = useState(false);

const selectedPlayerIds = squad.players.map((sp) => sp.player.id);

const playersByPosition = (position: Position) =>
  squad.players
    .filter((sp) => sp.position === position)
    .map((sp) => sp.player);

const addPlayerToPosition = (player: Player, position: Position) => {
  setSquad((prev) => {
    if (prev.players.some((sp) => sp.player.id === player.id)) return prev;
    return {
      ...prev,
      players: [...prev.players, { player, position }],
    };
  });
};

const removePlayer = (playerId: number) => {
  setSquad((prev) => ({
    ...prev,
    players: prev.players.filter((sp) => sp.player.id !== playerId),
  }));
};

const addTrainer = (trainer: Trainer) => {
  setSquad((prev) => {
    if (prev.trainers.some((t) => t.id === trainer.id)) return prev;
    return {
      ...prev,
      trainers: [...prev.trainers, trainer],
    };
  });
};

const removeTrainer = (trainerId: number) => {
  setSquad((prev) => ({
    ...prev,
    trainers: prev.trainers.filter((t) => t.id !== trainerId),
  }));
};
```

3.6 Zusätzliche Manuals, Skripts und weiteres

Für dieses Projekt sind keine zusätzlichen Manuals, Skripte oder weiteren Dokumente erforderlich.

Der vollständige Quellcode der Applikation wird separat als ZIP-Datei auf PKorg abgegeben.